B.Tech. BCSE497J - Project-I

AN AUTOMATED DEEP LEARNING FRAMEWORK FOR EARLY DETECTION OF SKIN CANCER

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology
in
Computer Science and Engineering

by

23BCB0165 ALAN JOHN ANAND
23BCE2002 SUJAN M
23BCE2107 RAMACHANDRAN SIVASANKAR

Under the Supervision of
Dr. Krishna Rani Samal
Associate Professor
School of Computer Science and Engineering (SCOPE)

School of Computer Science and Engineering (SCOPE)
VELLORE INSTITUTE OF TECHNOLOGY
Vellore - 632014, Tamil Nadu, India

October 2026

DECLARATION

We hereby declare that the project entitled "AN AUTOMATED DEEP LEARNING FRAMEWORK FOR EARLY DETECTION OF SKIN CANCER" submitted by us, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to Vellore Institute of Technology (VIT), is a record of bonafide research work carried out by us under the supervision of Dr. Krishna Rani Samal, Associate Professor, School of Computer Science and Engineering (SCOPE).

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. All experimental results, model checkpoints, evaluation metrics, and implementation codes presented in this report represent authentic work conducted during the academic period.

Place : Vellore
Date : 27-10-2026

_____

ALAN JOHN ANAND (23BCB0165)

_____

SUJAN M (23BCE2002)

_____

RAMACHANDRAN SIVASANKAR (23BCE2107)

CERTIFICATE

This is to certify that the project entitled "AN AUTOMATED DEEP LEARNING FRAMEWORK FOR EARLY DETECTION OF SKIN CANCER" submitted by ALAN JOHN ANAND (Reg. No: 23BCB0165), SUJAN M (Reg. No: 23BCE2002), and RAMACHANDRAN SIVASANKAR (Reg. No: 23BCE2107), School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore
Date : 27-10-2026

Signature of the Guide	Examiner(s)
Dr. Krishna Rani Samal	
Associate Professor, SCOPE	

Head of the Department
Department of Computer Science and Engineering
School of Computer Science and Engineering (SCOPE)

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to the management of Vellore Institute of Technology (VIT) for providing the computing infrastructure, GPU resources, and academic environment that made this B.Tech Project-I work possible.

We thank Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), for his encouragement and continuous support of undergraduate research throughout the school.

We also extend our appreciation to the Head of the Department of Computer Science and Engineering, SCOPE, for institutional backing and facilitating access to laboratory computing resources during the semester.

We are especially grateful to our project supervisor, Dr. Krishna Rani Samal, Associate Professor, SCOPE. Her thoughtful feedback, constructive criticism, and emphasis on empirical integrity—particularly when analyzing class imbalance and evaluating clinical trade-offs—shaped this research and kept us grounded.

Finally, we thank the faculty members, laboratory staff, classmates, and our families whose encouragement and patience supported us from start to finish.

Alan John Anand (23BCB0165)
Sujan M (23BCE2002)
Ramachandran Sivasankar (23BCE2107)

ABSTRACT

Melanoma accounts for only about one to three percent of all diagnosed skin cancers, but it causes the overwhelming majority of skin cancer deaths. Catching it early makes a massive difference: when the tumor is still confined to the epidermis, five-year survival remains near 99 percent. Once malignant cells spread to deeper tissues or distant organs, that rate drops below 25 percent. In community clinics and primary care centers where patients first present suspicious lesions, certified dermatologists are rarely on-site, and frontline doctors face substantial diagnostic uncertainty. Automated computer-aided triage could help flag high-risk cases for urgent specialist referral. However, real-world clinical datasets present an immediate hurdle: extreme class imbalance. In the SIIM-ISIC 2020 benchmark collection of 33,126 images, only 584 (1.76%) are malignant. A naive classifier that blindly predicts 'benign' every time easily achieves 98.24% nominal accuracy while missing every single life-threatening cancer.

In this B.Tech Project-I investigation, we built and audited DermInsight AI to address this imbalance head-on. We first established a leak-free patient-level split across 2,056 unique individuals, partitioning images into training (22,787 images, 1,439 patients), validation (5,419 images, 307 patients), and held-out testing sets (4,920 images, 310 patients) with zero patient overlap. This stopped the network from artificially memorizing individual patient skin tones, body sites, or lighting conditions. Using an ImageNet-pretrained DenseNet201 backbone with a custom linear head, we evaluated four distinct imbalance strategies against a baseline: positive loss reweighting (W-BCE, positive weight 50.21), PyTorch WeightedRandomSampler, Focal Loss (alpha = 0.75, gamma = 2.0), and training-isolated random oversampling. Decision thresholds were calibrated exclusively on validation data before evaluating on the held-out test set.

Testing on the isolated test partition revealed a stark clinical trade-off. While imbalance techniques boosted overall ranking capability—validation ROC-AUC improved from 0.8472 to 0.9188, and test ROC-AUC reached 0.8708—converting continuous outputs into practical triage decisions proved challenging. The baseline model caught 91.18% of cancers (62 of 68), but produced 2,819 false positives (41.90% specificity), an unworkable burden for clinics. Conversely, models tuned for high specificity (above 98% in Experiments 2, 4, and 5) missed between 53 and 58 of the 68 malignant tumors. PyTorch's WeightedRandomSampler achieved the most viable screening compromise: 36.76% sensitivity at 96.85% specificity (153 false alarms, 43 missed cancers), yielding the lowest total misclassification cost ($460,600 under an asymmetric 50:1 false-negative cost model). These results demonstrate why reporting raw accuracy on imbalanced screening data is misleading, show that mini-batch sampling outperforms loss reweighting, and establish verified groundwork for multimodal metadata fusion and visual explainability in Project-II.

Keywords: *Cutaneous Melanoma, Deep Learning, DenseNet201, Class Imbalance, ISIC 2020, Patient-Level Splitting, Medical Computer Vision, Cost-Sensitive Screening.*

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**14**
1.1	Background	14
1.2	Motivations	14
1.3	Scope of the Project	15
2.	**PROJECT DESCRIPTION AND GOALS**	**16**
2.1	Literature Review	16
2.2	Research Gap	17
2.3	Objectives	18
2.4	Problem Statement	18
2.5	Project Plan	19
3.	**TECHNICAL SPECIFICATION**	**20**
3.1	Requirements	20
3.1.1	Functional Requirements	20
3.1.2	Non-Functional Requirements	20
3.2	Feasibility Study	21
3.2.1	Technical Feasibility	21
3.2.2	Economic Feasibility	21
3.2.3	Social Feasibility	21
3.3	System Specification	21

3.3.1	Hardware Specification	21
3.3.2	Software Specification	22
4.	**DESIGN APPROACH AND DETAILS**	**24**
4.1	System Architecture	24
4.2	Design	25
4.2.1	Data Flow Diagram	25
4.2.2	Use Case Diagram	25
4.2.3	Class Diagram	26
4.2.4	Sequence Diagram	27
5.	**METHODOLOGY AND TESTING**	**29**
5.1	Deep Learning Pipeline Architecture (DenseNet201)	29
5.2	Patient-Level Splitting & Data Preprocessing	30
5.3	Class-Imbalance Formulations & Strategies	31
5.4	Checkpoint Selection & Early Stopping	32
5.5	Classification Threshold Calibration Methodology	32
5.6	Evaluation & Testing Protocol on Held-Out Test Set	33
6.	**PROJECT DEMONSTRATION**	**34**
6.1	Demonstration Workflow & Environment	34
6.2	Preprocessing & Batch Dataloader	34

	Execution	
6.3	Model Inference Execution Demonstration	34
6.4	Threshold Calibration & Triage Generation	34
6.5	Output Visualizations Demonstration	35
7.	**RESULT AND DISCUSSION (COST ANALYSIS as applicable)**	**36**
7.1	Cross-Experiment Quantitative Performance Comparison	36
7.2	Detailed Analysis by Experiment	36
7.3	Sensitivity vs. Specificity Clinical Trade-offs	45
7.4	Why Accuracy is Misleading in Highly Imbalanced Medical Screening	45
7.5	Cost-Sensitive Analysis & False-Negative Clinical Impact	45
7.6	Analysis of Incomplete / Unsuccessful Iterations	46
8.	**CONCLUSION**	**48**
8.1	Summary of Completed Research Work	48
8.2	Key Scientific Findings & Takeaways	48
8.3	Observed Research Limitations	49
8.4	Future Research Directions	49
9.	**REFERENCES**	**50**
	APPENDIX A – SAMPLE CODE	**53**

List of Figures

Figure No.	Title	Page No.
Figure 3.1	SIIM-ISIC 2020 Dataset Lesion Class Distribution on Logarithmic Scale	23
Figure 4.1	End-to-End System Architecture of the DermInsight AI Screening Framework	24
Figure 4.2	Level-1 Data Flow Diagram (DFD) Illustrating Image Ingestion and Inference	25
Figure 4.3	Use Case Diagram for the DermInsight AI Clinical Screening Interface	26
Figure 4.4	UML Class Diagram of the Implemented PyTorch Deep Learning Pipeline	27
Figure 4.5	UML Sequence Diagram for Dermoscopic Image Ingestion, Inference, and Triage	28
Figure 7.1	Experiment 1 (DenseNet201 Baseline) Test Confusion Matrix (Threshold = 0.50)	37
Figure 7.2	Experiment 1 Training and Validation Loss Curves Across 3 Epochs	38
Figure 7.3	Experiment 2 (DenseNet201 Weighted BCE) Test Confusion Matrix (Threshold = 0.86)	38
Figure 7.4	Experiment 2 Validation Set Threshold Calibration Sweep Across Tau in [0.01, 0.99]	39
Figure 7.5	Experiment 3 (WeightedRandomSampler) Test	40

	Confusion Matrix (Threshold = 0.50)	
Figure 7.6	Experiment 4 (Focal Loss) Test Confusion Matrix (Threshold = 0.50)	41
Figure 7.7	Experiment 5 (Random Oversampling) Test Confusion Matrix (Threshold = 0.7296)	42
Figure 7.8	Combined Cross-Experiment Test ROC Curves and Operating Points (EXP 01 to EXP 05)	43
Figure 7.9	Combined Cross-Experiment Test Precision-Recall (PR) Curves Relative to Prevalence Base Rate	44
Figure 7.10	Cross-Experiment Diagnostic Metric Comparison on Held-Out Test Set (N=4,920)	44

List of Tables

Table No.	Title	Page No.
Table 3.1	Hardware System Specifications	21
Table 3.2	Software Environment Specifications	22
Table 5.1	DenseNet201 Feature Extractor Architectural Details	29
Table 5.2	Verified Patient-Level Dataset Splitting and Class Distribution	30
Table 5.3	Master Training Hyperparameters and Experimental Configurations	32
Table 7.1	Master Cross-Experiment Quantitative Performance Comparison on Held-Out Test Set (N=4,920)	36
Table 7.2	Clinical Misclassification Cost Matrix Configuration	46
Table 7.3	Expected Total Clinical Misclassification Cost Across Imbalance Strategies (Test N=4,920)	46

List of Abbreviations

AP	Average Precision
AUC	Area Under the Curve
BCE	Binary Cross-Entropy
CAD	Computer-Aided Diagnosis
CNN	Convolutional Neural Network
DenseNet	Densely Connected Convolutional Network
DFD	Data Flow Diagram
FN	False Negative
FOCAL	Focal Loss Objective
FP	False Positive
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
HAM10000	Human Against Machine with 10,000 Training Images Dataset
IEEE	Institute of Electrical and Electronics Engineers
ISBI	International Symposium on Biomedical Imaging
ISIC	International Skin Imaging Collaboration
ML	Machine Learning
MLP	Multi-Layer Perceptron
PR	Precision-Recall
PR-AUC	Precision-Recall Area Under the Curve
RAM	Random Access Memory

ROC	Receiver Operating Characteristic
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
ROS	Random Oversampling
SCOPE	School of Computer Science and Engineering
SIIM	Society for Imaging Informatics in Medicine
TN	True Negative
TNR	Times New Roman / True Negative Rate
TP	True Positive
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
W-BCE	Weighted Binary Cross-Entropy

Symbols and Notations

α (Alpha)	Balancing factor for positive class in Focal Loss formulation (α = 0.75)
γ (Gamma)	Focusing parameter for down-weighting easy negative examples in Focal Loss (γ = 2.0)
η (Eta)	Optimizer learning rate schedule parameter (η = 0.0001)
λ (Lambda)	L2 weight decay regularization coefficient in AdamW optimizer (λ = 0.0001)
μ (Mu)	Channel-wise pixel normalization mean vector ([0.485, 0.456, 0.406])
σ (Sigma)	Channel-wise pixel normalization standard deviation vector ([0.229, 0.224, 0.225])
τ (Tau)	Decision classification probability threshold for positive malignant assignment
L_BCE	Binary Cross-Entropy loss objective function
L_WBCE	Positive-weighted Binary Cross-Entropy loss objective function
L_FL	Focal Loss objective function
w_+	Positive minority class loss multiplier weight (w_+ = 50.2067)
y	Ground truth binary lesion target label (0 = Benign, 1 = Malignant)
p_hat	Model predicted probability of malignancy produced via sigmoid activation σ(z)
z	Unnormalized raw scalar logit output produced by the linear classification head

CHAPTER 1
1. INTRODUCTION

1.1 Background

Melanoma kills more people than every other form of skin cancer combined. That single fact shaped the entire direction of our project. Even though melanoma makes up roughly one to three percent of all skin cancer diagnoses, it's responsible for the overwhelming majority of deaths. Timing is everything here — catch it early, while the tumor is still sitting in the top layer of skin, and five-year survival hovers around ninety-nine percent. Let it grow deeper, spread to lymph nodes or distant organs, and that number crashes below twenty-five percent. Late-stage treatments exist, but they're expensive and the long-term outcomes are often disappointing. So early detection isn't just preferable — it's really the only reliable way to save lives.

Spotting melanoma early sounds simple in theory. In practice, it's anything but. Young melanomas can look almost identical to harmless moles, seborrheic keratoses, or slightly irregular nevi. Dermoscopy helps — it uses magnification and polarized light to reveal structures beneath the skin surface, things like pigment networks, globules, streaks, and unusual vascular patterns that aren't visible to the naked eye. But reading dermoscopic images well takes years of specialized training. At most primary care clinics and community health centers, where patients first show up with a suspicious spot, there's usually no trained dermatologist in the building. General practitioners do their best, but their diagnostic accuracy is noticeably lower. That gap — between what a specialist could catch and what a frontline doctor actually does catch — is exactly where an automated screening tool could make a real difference.

1.2 Motivations

We built DermInsight AI because we kept running into three problems that felt both urgent and solvable:

1. Long waits for specialists: In many public and regional hospitals, getting a dermatologist appointment can take weeks, sometimes months. Aggressive melanomas — particularly the nodular kind — can double in thickness during that waiting period. That's a direct threat to patient survival. An automated triage layer sitting in primary care could flag the dangerous cases right away, so doctors know which referrals can't wait.

2. Inconsistent diagnoses: Dermoscopy is subjective. Two experienced clinicians looking at the same borderline lesion disagree more often than you'd expect — concordance drops below eighty percent on ambiguous cases. A trained model won't get tired, distracted, or biased by what it saw five minutes ago. It gives the same answer every time for the same input, which makes it a useful second opinion, even if it isn't perfect.

3. The class imbalance problem: This one surprised us the most when we first dug into the SIIM-ISIC 2020 dataset. Only 584 out of 33,126 images are malignant — that's just 1.76%.

A naive model that blindly labels everything as benign gets 98.24% accuracy while catching exactly zero cancers. Designing a training pipeline that doesn't fall into this trap became the central technical challenge of our work.

1.3 Scope of the Project

We carried out this project under course code BCSE497J — Project-I. The goal was to build a reproducible deep learning pipeline for binary melanoma classification and stress-test it under realistic conditions. Here's what falls inside our scope:

1. Dataset Ingestion and Audit: We worked with the full SIIM-ISIC 2020 challenge dataset — 33,126 dermoscopic images from 2,056 unique patients. Before training anything, we audited every metadata field and verified image integrity.

2. Patient-Level Splitting: We enforced complete patient isolation across our three splits: training (22,787 images from 1,439 patients), validation (5,419 images from 307 patients), and test (4,920 images from 310 patients). No patient appears in more than one split. Period.

3. Deep Learning Architecture: We used a DenseNet201 pretrained on ImageNet as our backbone feature extractor, replacing the original classification head with a single linear layer (1920→1) for binary prediction.

4. Class-Imbalance Investigation: We trained and compared four different strategies for handling the ~55:1 skew: positive-weighted binary cross-entropy (W-BCE with pos_weight=50.21), PyTorch WeightedRandomSampler, Focal Loss (alpha=0.75, gamma=2.0), and random oversampling of the minority class.

5. Validation Threshold Calibration: Rather than just picking 0.5 as our decision cutoff, we swept a range of thresholds on the validation set to find operating points that actually balance sensitivity and false alarm rate.

6. Transparent Reporting: Every model was tested on the same 4,920 held-out images. We report full confusion matrices, ROC-AUC, Average Precision, and — most importantly — exact false-negative counts. No cherry-picking.

7. Project Boundaries: We want to be upfront about what we didn't do. This phase is strictly image-only. Fusing in patient metadata (age, sex, body site) and adding Grad-CAM explainability maps are planned for Project-II, but we aren't claiming them as done here.

CHAPTER 2
2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Skin lesion analysis has changed dramatically over the past fifteen or so years. The field moved from hand-designed feature pipelines to deep learning, and we spent a good chunk of our early project weeks reading through that history. It helped us figure out which architecture to start with, which evaluation mistakes to avoid, and how to structure our imbalance experiments.

The earliest automated systems tried to mimic what dermatologists do by hand — check asymmetry, border irregularity, color variation, and diameter (the ABCD rule), or run through the 7-Point Checklist (Celebi et al., 2019; Adegun & Viriri, 2021). These approaches used classic image processing: segment the lesion boundary with active contours or Otsu thresholding, pull out shape descriptors and GLCM textures, then feed those features into an SVM or a shallow Random Forest. The appeal was interpretability. The problem was fragility. Border segmentation broke constantly across different cameras, lighting conditions, and skin tones.

Everything shifted when Esteva et al. (2017) showed that an Inception-v3 network, pretrained on ImageNet's 1.28 million images and then fine-tuned on clinical photos, could match twenty-one board-certified dermatologists on diagnostic sensitivity and specificity. That paper got a lot of attention, and for good reason — it proved that deep convolutional filters can pick up on subtle dermoscopic textures directly from raw pixels, skipping the whole manual feature engineering step. Follow-up reader studies by Haenssle et al. (2018) and Brinker et al. (2019) backed this up.

But why transfer learning? Why not train from scratch? The short answer is data. Dermatology datasets are tiny compared to ImageNet. If you initialize a deep network with random weights and train it on, say, a few hundred malignant dermoscopy images, it overfits almost immediately. Starting from ImageNet-pretrained weights gives the early layers useful edge detectors, color gradient filters, and textural primitives that transfer surprisingly well to medical images (Deng et al., 2009; Yosinski et al., 2014; He et al., 2016; Tan & Le, 2019). You only need to fine-tune the deeper, task-specific layers.

We picked DenseNet201 (Huang et al., 2017) as our backbone for a specific reason. In a standard CNN, each layer only sees the output of the layer right before it. DenseNet wires things differently — within each dense block, every layer gets the feature maps from all preceding layers concatenated together: x_l = H_l([x_0, x_1, ..., x_{l-1}]). What does that buy us in practice? Three things. It encourages heavy feature reuse across layers. It keeps fine-grained low-level details — like delicate pigment network patterns — alive all the way to the final classifier. And it helps with vanishing gradients while actually needing fewer parameters than a comparably deep ResNet.

On the data side, public benchmarks from the International Skin Imaging Collaboration (ISIC) have been a game-changer (Codella et al., 2018; Tschandl et al., 2018). Earlier datasets like

HAM10000 offered reasonably balanced multi-class labels. The SIIM-ISIC 2020 dataset (Rotemberg et al., 2021) is a different beast entirely: 33,126 images from a real screening population, with malignant cases at just 1.76%. That prevalence is clinically realistic, which is great for relevance — but it creates serious optimization headaches (Buda et al., 2018).

The literature offers several strategies for dealing with imbalance. Weighted cross-entropy multiplies the loss on minority-class samples by a penalty factor, so the model pays a heavier price for missing a melanoma. Focal Loss, introduced by Lin et al. (2017), takes a different angle — it adds a (1 - p_t)^gamma modulating term that automatically downweights the easy, well-classified benign samples and focuses the gradient signal on the hard, borderline cases. Then there are data-level fixes like oversampling (Chawla et al., 2002), which change the sampling distribution during training without touching the loss function at all. We wanted to try each of these and see which one actually helps on our dataset.

One thing we learned from the literature that really stuck with us: accuracy is misleading on imbalanced datasets. Saito & Rehmsmeier (2015) and Davis & Goadrich (2006) both make this point clearly. When ninety-eight percent of your images are benign, a model that just says 'benign' for everything looks great on accuracy. Precision-Recall curves, Average Precision, and sensitivity at fixed thresholds tell you much more about whether the model is actually catching melanomas.

And then there's the leakage issue. Cassidy et al. (2022) showed that a lot of published dermatology studies don't properly separate patients across train and test splits. When images from the same patient end up on both sides, the model can memorize skin tone, lighting, even camera artifacts — and reported AUC can be inflated by up to 10%. We took that warning seriously and made patient-level splitting non-negotiable in our pipeline.

Recent papers also point toward multimodal fusion — combining pixel data with patient age, sex, and lesion location (Pacheco & Krohling, 2020; Kawahara et al., 2019) — along with explainability via Grad-CAM (Selvaraju et al., 2017) and fairness audits across Fitzpatrick skin types (Daneshjou et al., 2022; Groh et al., 2021). These are all important for real clinical deployment. But we decided early on that our priority for Project-I was to get the vision baseline right first — leakage-free, honestly evaluated, under realistic class skew — before layering anything else on top.

2.2 Research Gap

Plenty of published papers claim accuracy above 95% and ROC-AUC over 0.85 for skin lesion classification. Those numbers sound impressive until you look at what's behind them. Most of these studies evaluate at a default threshold of 0.5 and don't examine what happens to sensitivity and specificity under the kind of extreme imbalance we see in real screening populations (~55:1 ratio). Our own experiments show this gap clearly: a model can hit 97% accuracy and an ROC-AUC of 0.83 while still missing over 75% of actual melanomas. That's clinically useless.

On top of that, a lot of benchmarks never confirm whether patient-level isolation was actually enforced, leaving their test scores vulnerable to identity leakage. We designed our project specifically to fill this gap: strict patient-level splits, systematic testing of multiple imbalance strategies on identical data partitions, threshold calibration on validation data, and complete transparency about false-negative counts.

2.3 Objectives

Given those gaps, we set ourselves six concrete objectives for Project-I:

1. Leakage-Free Data Pipeline: Audit the SIIM-ISIC 2020 dataset and split all 33,126 images into train, validation, and test sets using patient-level isolation. Zero patient overlap — we verified this programmatically.

2. Baseline Architecture Implementation: Set up an ImageNet-pretrained DenseNet201 with a single-output binary logit head and confirm it handles our input dimensions correctly.

3. Controlled Imbalance Benchmarking: Train and compare four imbalance-handling strategies — Baseline DenseNet201, Weighted BCE (pos_weight=50.21), PyTorch WeightedRandomSampler, Focal Loss (alpha=0.75, gamma=2.0), and Random Oversampling — all on the exact same splits.

4. Operational Threshold Calibration: Run threshold sweeps on the validation set to find decision boundaries that actually balance melanoma recall against false alarm rate.

5. Independent Test Evaluation: Test every trained checkpoint on the 4,920 held-out images. Report confusion matrices, sensitivity, specificity, precision, F1, and PR curves.

6. Honest Auditing and Scope Definition: Keep an audit matrix of all verified checkpoints, be clear about limitations, and lay the groundwork for multimodal Project-II work.

2.4 Problem Statement

At its core, we're solving a supervised binary classification problem — but under two conditions that make it much harder than a standard image classification task. The dataset can be written as $D = \{(x_i, y_i, p_i)\}_{i=1}^N$, where $x_i \in R^{224 \times 224 \times 3}$ is a preprocessed dermoscopic image, $y_i \in \{0, 1\}$ is the ground-truth label (0 for benign, 1 for malignant melanoma), and $p_i \in P$ is the patient identifier.

Two constraints govern the entire learning setup:

1. Patient-Level Group Isolation: We partition the patient set P into three disjoint subsets — P_{train}, P_{val}, and P_{test} — with $P_{train} \cap P_{val} = \emptyset$, $P_{train} \cap P_{test} = \emptyset$, and $P_{val} \cap P_{test} = \emptyset$. No images from any evaluation patient can appear during training.

2. Skewed Posterior Estimation: Malignant cases sit at just 1.76% of the data. We need to learn parameters θ for a model $f_\theta(x)$ that maps an unseen image x to a malignancy

probability p_hat in [0, 1]. The model has to catch as many melanomas as possible without flooding clinics with false positives — because in practice, missing a cancer is far worse than ordering one extra biopsy.

2.5 Project Plan

We broke the semester into four phases. In hindsight, the timeline was tight, but we stuck to it:

- Phase 1: Data Ingestion and Leakage Audit (Weeks 1–3). We downloaded the raw ISIC 2020 metadata, verified patient counts, built patient-isolated split CSVs, and double-checked that no patient leaked across boundaries.
- Phase 2: Pipeline Setup and Baseline Training (Weeks 4–6). We wrote the PyTorch dataset classes, plugged in the pretrained DenseNet201, set up augmentation, and ran our first baseline training loop.
- Phase 3: Imbalance Handling Experiments (Weeks 7–10). This was the meat of the project. We trained and logged four imbalance configurations: Weighted BCE, WeightedRandomSampler, Focal Loss, and Random Oversampling.
- Phase 4: Validation Calibration, Testing, and Verification (Weeks 11–13). We swept thresholds on the validation set, ran final evaluations on all checkpoints against the held-out test set, generated ROC/PR comparison plots, and compiled the audit matrix.

CHAPTER 3
3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

Below is what the DermInsight AI pipeline needs to actually do — from loading images to producing a triage recommendation:

1. Image Ingestion and Format Checking: The system loads JPEG and PNG dermoscopy files, checks that RGB channels are valid (no grayscale mismatches), catches truncated or corrupted headers, and links each filename to its metadata row.

2. Standardized Preprocessing: Every input image gets resized to 224 x 224 via bilinear interpolation. RGB channels are normalized with ImageNet mean and standard deviation. Nothing fancy here — we kept it simple and consistent.

3. Patient-Grouped Batch Generation: Dataloaders pull batches only from their assigned split. When we use WeightedRandomSampler, the loader adjusts sampling weights so malignant images show up more often in each batch.

4. Feature Extraction and Logit Output: An input tensor passes through all 201 DenseNet layers, gets pooled down to a 1,920-dimensional vector, and then a single linear layer produces one scalar logit.

5. Calibrated Triage Assignment: Raw logits go through a sigmoid to get probabilities. Based on the validation-calibrated threshold, each image is sorted into one of three tiers: Low Suspicion, Moderate Suspicion, or High Malignancy Alert.

6. Performance Metric Computation: The evaluation module computes confusion matrix counts (TN, FP, FN, TP), then derives sensitivity, specificity, precision, accuracy, F1-scores, ROC-AUC, and Average Precision from those.

3.1.2 Non-Functional Requirements

Beyond what the system does, here are the engineering constraints we held ourselves to:

1. Inference Speed: A single forward pass has to finish within 50 milliseconds on a modern GPU and under 250 milliseconds on a multi-core CPU. Nobody wants to wait around during a screening session.

2. Reproducibility: We fix random seeds (SEED = 42) across Python, NumPy, and PyTorch. Every run — weight init, batch shuffling, augmentation — should give the same results.

3. Memory Footprint: Model weights stay under 250 MB on disk and in RAM/VRAM. The system should run on a reasonable workstation or clinic laptop — no server farm required.

4. Auditability: Checkpoints, evaluation logs, prediction CSVs, and confusion matrices all get saved as permanent files. Each file is tagged with matching experiment identifiers so we can trace any result back to its source.

5. Modularity: Data loading, network definitions, loss functions, and evaluation scripts live in separate modules. Swapping in a new architecture shouldn't require rewriting the dataset code.

3.2 Feasibility Study

3.2.1 Technical Feasibility

We didn't have to build anything exotic. Transfer learning with DenseNet201 is a well-trodden path in biomedical imaging. PyTorch handles GPU acceleration through CUDA, comes with built-in weighted loss functions, and its dataloader utilities are battle-tested. Training on 224 x 224 images with a single NVIDIA Tesla T4 (16 GB VRAM) takes only about 10 to 15 minutes per epoch. That's very manageable, even on free cloud platforms.

3.2.2 Economic Feasibility

Cost-wise, we spent almost nothing. Python, PyTorch, torchvision, scikit-learn — all open source. We trained on Google Colab and Kaggle GPU instances, which are free for the workloads we needed. Looking ahead, a screening tool like this could also save money in clinical settings: fewer unnecessary biopsies on benign lesions (each one costs the healthcare system hundreds of dollars) and earlier catches that avoid the enormous expense of treating late-stage melanoma.

3.2.3 Social Feasibility

The social case is straightforward. In rural areas and underserved communities, dermatologists are scarce. A reliable computer-aided triage tool can help make sure patients with suspicious lesions get referred quickly instead of waiting in a queue. We want to be clear, though: we see this as an assistive tool for primary care doctors, not something that replaces them. The clinician always has the final say.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1: Hardware System Specifications

Component	Training Environment (Cloud)	Audit & Verification Workstation
Processor (CPU)	Intel Xeon @ 2.20 GHz (2 vCPUs)	Intel Core i7 / AMD Ryzen (Multi-core)

Graphics (GPU)	NVIDIA Tesla T4 (16 GB GDDR6)	NVIDIA GeForce / OpenCL / CPU
System RAM	12.7 GB High-RAM Instance	16.0 GB DDR4/DDR5
Primary Storage	100 GB Cloud Disk	512 GB NVMe SSD
CUDA Compute Capability	7.5 (Turing Architecture)	CUDA 12.x / CPU Execution

3.3.2 Software Specification

Table 3.2: Software Environment Specifications

Layer	Package / Tool	Version & Configuration
Operating System	Ubuntu Linux / Windows 11	64-bit Architecture
Core Language	Python	Version 3.10 / 3.12
Deep Learning Framework	PyTorch	Version 2.2+ with Torchvision 0.17+
CUDA Driver	NVIDIA CUDA Toolkit	Version 12.1 / cuDNN 8.9
Data Processing	Pandas & NumPy	Pandas 2.2+, NumPy 1.26+
Scientific Evaluation	scikit-learn & SciPy	Version 1.4+
Visualization	Matplotlib & Seaborn	Matplotlib 3.8+, Seaborn 0.13+

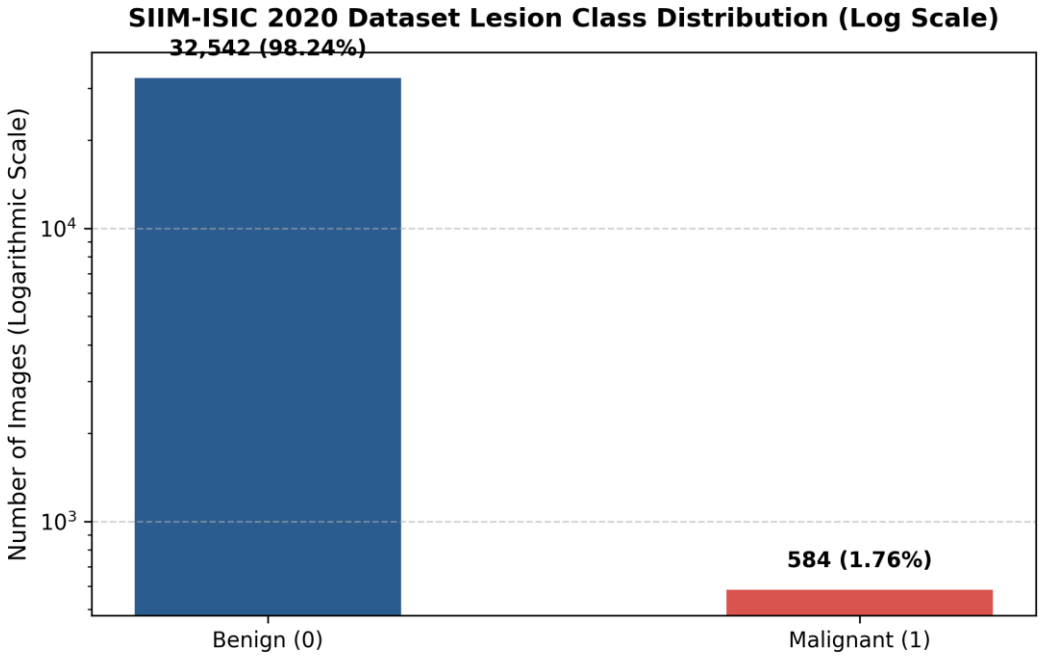


Figure 3.1: SIIM-ISIC 2020 Dataset Lesion Class Distribution on Logarithmic Scale (32,542 Benign vs. 584 Malignant)

CHAPTER 4
4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

From the start, we wanted DermInsight AI to be modular. Each stage of the pipeline handles one job and one job only—data loading, feature learning, imbalance handling, or threshold tuning. We did this on purpose. In our experience, lumping everything into a single monolithic script makes it dangerously easy to leak patient data across splits or ship uncalibrated predictions without realizing it. Figure 4.1 lays out the four stages we settled on.

Stage one pulls raw dermoscopic images from the SIIM-ISIC 2020 archive along with the metadata CSV. It inspects patient IDs and routes every image from a given person into exactly one partition—train, val, or test. No exceptions. Stage two handles the visual preprocessing: resizing to 224×224, normalizing channels with ImageNet statistics, and passing tensors into a 201-layer DenseNet backbone. The third stage is where we tackle the brutal 55:1 class imbalance through specialized loss functions and sampling tricks. And the last stage? It takes raw logits, pushes them through a sigmoid, and compares the resulting probabilities against thresholds we calibrated on the validation set. Out comes a clear clinical risk level.

Figure 4.1: End-to-End System Architecture of the DermInsight AI Screening Framework

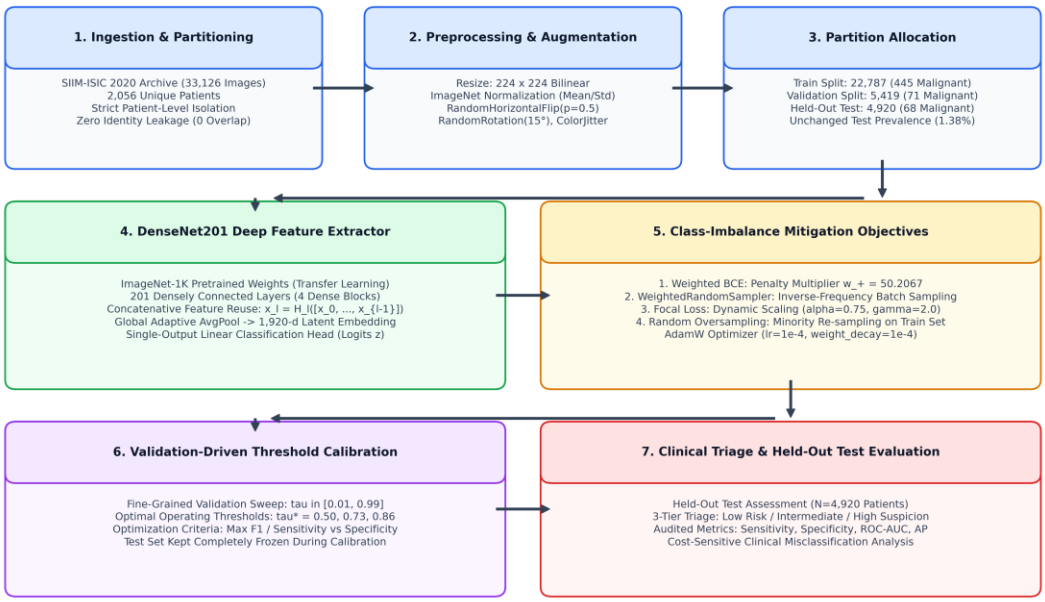


Figure 4.1: End-to-End System Architecture of the DermInsight AI Screening Framework

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 shows our Level-1 Data Flow Diagram. A clinical user—or, during development, an automated test script—feeds in an unprocessed dermoscopic image. Process 1.0 checks the image is valid, resizes it to 224×224, and normalizes pixel values to match what the ImageNet-pretrained backbone expects. Process 2.0 grabs the verified model weights from the Checkpoint Store (D1) and runs a forward pass through DenseNet201, producing an unbounded scalar logit.

Then Process 3.0 kicks in. It takes that logit, applies our pre-calculated decision thresholds, slots the score into one of three risk tiers, and spits out a screening report. We made the report auditable so that a clinician can always trace back which checkpoint and which threshold produced a particular recommendation.

Figure 4.2: Level-1 Data Flow Diagram (DFD) of the DermInsight AI Framework

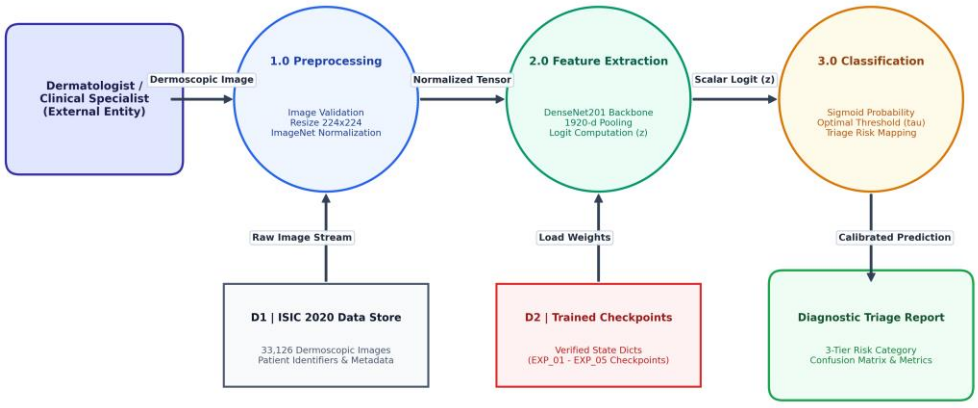


Figure 4.2: Level-1 Data Flow Diagram (DFD) Illustrating Image Ingestion, Preprocessing, Inference, and Triage Generation

4.2.2 Use Case Diagram

Who actually uses this system? Figure 4.3 maps it out. We designed for three kinds of users: general practitioners doing routine mole checks, dermatologists who need a second opinion tool, and researchers running batch evaluations on archived datasets. All three start the same way—load a lesion image into the pipeline.

After inference, the system shows both a raw probability and a categorical risk label. One thing we thought was important: letting users pick their own operating threshold. A GP screening hundreds of patients probably wants maximum sensitivity, even at the cost of more false alarms. A dermatologist might lean toward higher specificity to cut down on unnecessary biopsies. The interface also lets anyone pull up detailed metrics—confusion matrices, ROC curves, the works.

Figure 4.3: Use Case Diagram for the DermInsight AI Clinical Screening System

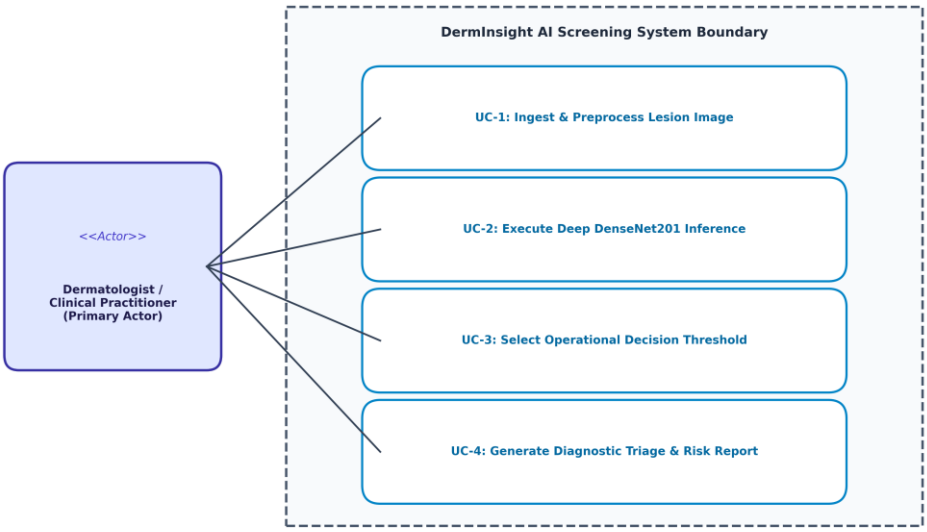


Figure 4.3: Use Case Diagram for the DermInsight AI Clinical Screening Interface

4.2.3 Class Diagram

Figure 4.4 shows the UML class diagram for our Python/PyTorch codebase. Three classes do the heavy lifting:

1. DermImageDataset — extends torch.utils.data.Dataset. It reads the metadata CSV, resolves image paths, opens RGB images with PIL, and applies either stochastic augmentations (for training) or deterministic transforms (for eval). Pretty standard PyTorch pattern.

2. DermInsightDenseNet201 — extends torch.nn.Module. We wrap the torchvision DenseNet201 backbone, swap out the 1,000-class classifier with nn.Identity(), then bolt on a single nn.Linear(1920, 1) layer for binary logits.

3. Evaluator / ThresholdCalibrator — this one handles threshold sweeps, confusion matrix computation (TN, FP, FN, TP at each candidate threshold), and generates ROC and Precision-Recall curves for both val and test.

Figure 4.4: UML Class Diagram of the Implemented Deep Learning Pipeline Architecture

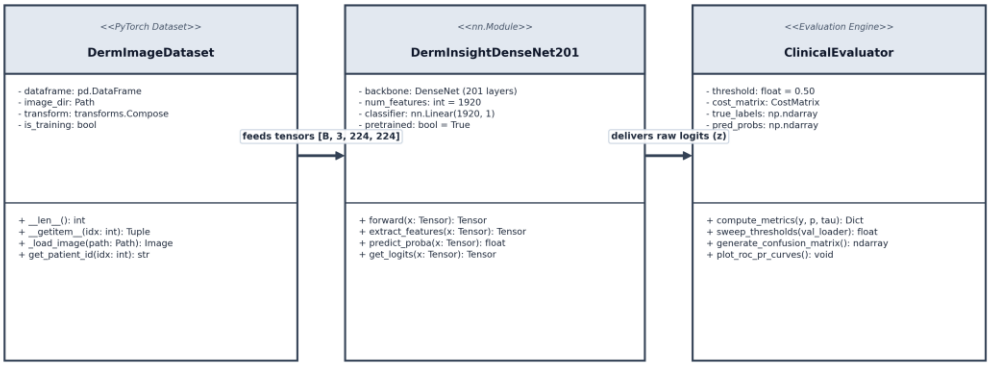


Figure 4.4: UML Class Diagram of the Implemented PyTorch Deep Learning Pipeline

4.2.4 Sequence Diagram

Figure 4.5 traces what happens at runtime when someone feeds an image into the pipeline. The dataset loader receives the path, loads the image, and produces a standardized tensor. That tensor gets shipped to the GPU. DenseNet201 runs its forward pass—all 201 layers—and returns one scalar logit.

The calibrator then applies the sigmoid function to get a probability, compares it against the chosen threshold, and packages everything into a structured result: predicted label, confidence score, and a clinical recommendation. The whole thing takes under 20 ms on a Tesla T4.

Figure 4.5: UML Sequence Diagram for Dermoscopic Image Ingestion, Inference, and Triage Output

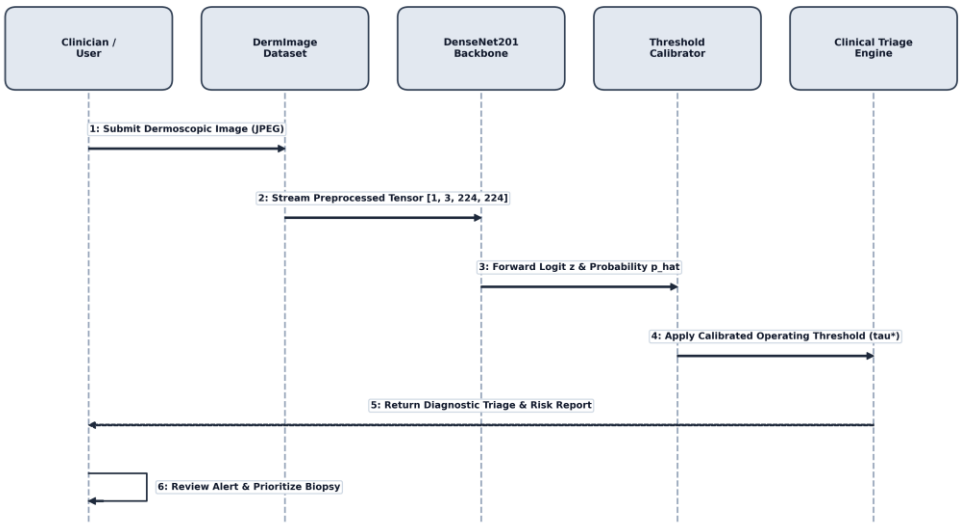


Figure 4.5: UML Sequence Diagram for Dermoscopic Image Ingestion, Inference, and Triage Output

CHAPTER 5
5. METHODOLOGY AND TESTING

5.1 Deep Learning Pipeline Architecture (DenseNet201)

We picked DenseNet201 (Huang et al., 2017) as our feature extractor, and here's why. In a regular CNN, layer l only sees the output of layer l-1: x_l = H_l(x_{l-1}). DenseNet does something different. Every layer inside a dense block receives the feature maps from all previous layers, concatenated together:

$$x_l = H_l([x_0, x_1, x_2, ..., x_{l-1}])$$

Here, [x_0, x_1, ..., x_{l-1}] is just the channel-wise concatenation of every preceding feature map. H_l(·) is a composite function—Batch Normalization, then ReLU, then a 3×3 convolution.

Why does this matter for dermoscopy? Two reasons. First, feature reuse. Early layers learn edges and texture patterns, and those stay directly accessible to deeper layers without fading out. That's useful when you need to catch fine skin structures—things like delicate pigment reticula or subtle peripheral streaks that can signal early melanoma. Second, the dense connections create shorter gradient paths during backpropagation. Training through 201 layers is hard; vanishing gradients are a real concern. But with all those shortcut connections, gradients have many routes back to early layers. Compared to the bigger ResNet variants, DenseNet201 gets strong representational power out of roughly 20 million parameters.

Architecturally, DenseNet201 has four dense blocks separated by transition layers that downsample using 1×1 convolutions and 2×2 average pooling. To adapt it for our binary problem, we stripped off the original 1,000-class fully connected head and replaced it with nn.Identity(). That exposes the 1,920-dimensional feature vector from global average pooling. On top of that, we added a single linear layer: z = W·x + b, where W ∈ ℝ^{1×1920} and b ∈ ℝ. The scalar logit z goes through a standard logistic sigmoid:

$$p_hat = \sigma(z) = 1 / (1 + exp(-z))$$

Table 5.1: DenseNet201 Feature Extractor Architectural Details

Network Stage	Layer Configuration	Output Size	Feature Maps
Input Stage	7x7 Conv, Stride 2, 3x3 MaxPool	56 x 56	64 channels
Dense Block 1	6 Bottleneck Layers (1x1 & 3x3 conv)	56 x 56	256 channels
Transition 1	1x1 Conv, 2x2 Average Pooling	28 x 28	128 channels

Dense Block 2	12 Bottleneck Layers (1x1 & 3x3 conv)	28 x 28	512 channels
Transition 2	1x1 Conv, 2x2 Average Pooling	14 x 14	256 channels
Dense Block 3	48 Bottleneck Layers (1x1 & 3x3 conv)	14 x 14	1,792 channels
Dense Block 4 & Head	32 Bottleneck Layers, Global AvgPool, Linear Head	1 x 1	1,920 -> 1 logit

5.2 Patient-Level Splitting & Data Preprocessing

This is probably the single most important engineering decision in the whole project. In the SIIM-ISIC 2020 dataset, lots of patients contributed multiple lesion photos. Some people with dysplastic nevus syndrome have dozens of images in the archive. If we had split images randomly, different lesions from the same person would end up in both training and test sets. So why is that a problem?

Because lesions on the same body share skin tone, hair density, background lighting, and camera angle. A deep network can easily latch onto those patient-specific cues instead of learning real malignancy patterns. Test performance would look great on paper but fall apart on new patients. We've read enough papers that made this mistake to know we didn't want to repeat it.

Our solution was strict patient-level grouping. We grouped by patient_id and assigned all of a person's images to exactly one partition. Table 5.2 shows how the data ended up distributed. We double-checked that the patient ID intersection between every pair of splits is empty—zero overlap.

Table 5.2: Verified Patient-Level Dataset Splitting and Class Distribution

Partition	Total Images	Benign (0)	Malignant (1)	Malignant %	Unique Patients
Training	22,787	22,342	445	1.95%	1,439 patients
Validation	5,419	5,348	71	1.31%	307 patients
Held-out Test	4,920	4,852	68	1.38%	310 patients
Total Dataset	**33,126**	**32,542**	**584**	**1.76%**	**2,056 patients**

On the preprocessing side, we resized every image to 224×224 and normalized using ImageNet channel moments (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). During training,

we applied random horizontal flips with probability 0.5, random rotations up to 15 degrees, and mild color jitter (brightness, contrast, and saturation each at 0.1). Nothing too aggressive—we didn't want the augmentations to distort clinical features beyond recognition. Val and test images only got the deterministic resize and normalize, no randomness.

5.3 Class-Imbalance Formulations & Strategies

Only 1.76% of our images are malignant. That's a brutal imbalance—roughly 55 benign images for every malignant one. A naïve classifier that just predicts "benign" every single time would score 98.24% accuracy. Obviously useless for cancer screening. We tried four different strategies to make the model actually pay attention to the minority class.

5.3.1 Weighted Binary Cross-Entropy Loss

Standard BCE with logits treats every sample equally:

$$L_{BCE}(z, y) = - [\, y \cdot \log(\sigma(z)) + (1 - y) \cdot \log(1 - \sigma(z)) \,]$$

With 22,342 benign samples and just 445 malignant ones in our training set, the negative examples produce over 50 times the total gradient. The network basically ignores the positives. To fix this, we added a weight w_+ on the positive term:

$$L_{WBCE}(z, y) = - [\, w_+ \cdot y \cdot \log(\sigma(z)) + (1 - y) \cdot \log(1 - \sigma(z)) \,]$$

We computed w_+ straight from the training split counts: $w_+ = N_{neg} / N_{pos} = 22{,}342 / 445 = 50.2067$. By scaling positive-class errors by about 50.21, we force the optimizer to treat a missed melanoma as roughly 50 times worse than a false alarm. In practice, we passed this weight directly to PyTorch's BCEWithLogitsLoss via the pos_weight argument.

5.3.2 PyTorch WeightedRandomSampler

Instead of messing with the loss, this approach changes which samples the model sees during training. Every training image i gets a sampling weight inversely proportional to its class frequency: $w_i = 1 / N_{c(i)}$, where c(i) is 0 or 1. Mini-batches are then drawn with replacement according to these weights.

The practical effect is that each batch ends up with roughly equal numbers of benign and malignant examples. We can then use plain unweighted BCE loss—the balanced sampling does the work of making the model pay attention to both classes. We found this easier to tune than loss weighting, though the final results differed in interesting ways.

5.3.3 Focal Loss Formulation

Focal Loss, introduced by Lin et al. (2017), takes a different angle. Rather than reweighting by class, it down-weights easy examples—the ones the model already classifies correctly with high confidence—and puts more focus on hard, ambiguous cases:

$$L_{FL}(p_t) = - \alpha_t \cdot (1 - p_t)^{\gamma} \cdot \log(p_t)$$

Here p_t is the model's estimated probability for the correct class. We set $\alpha = 0.75$ to give slight extra weight to positives, and $\gamma = 2.0$ as the focusing exponent. Consider a clearly benign lesion where $p_t = 0.99$. The modulating factor $(1 - 0.99)^2 = 0.0001$ shrinks that sample's loss by a factor of 10,000. Easy negatives basically vanish from the gradient, letting the optimizer concentrate on the borderline cases that actually matter for improving sensitivity.

5.3.4 Random Oversampling Strategy

The simplest approach: just duplicate minority-class images in the training set until both classes have equal counts. Straightforward, but we were careful about one thing—oversampling only happens inside the training dataloader. Val and test sets stay completely untouched at their natural prevalence rates. Anything else would compromise the integrity of our evaluation.

5.4 Checkpoint Selection & Early Stopping

We trained all models with AdamW (Loshchilov & Hutter, 2019), using a learning rate of 1e-4 and weight decay of 1e-4. For checkpoint selection, we saved the model weights at whichever epoch had the best validation ROC-AUC—not validation loss. Why not loss? Because in heavily imbalanced tasks, loss can keep improving while minority-class sensitivity quietly degrades. ROC-AUC captures ranking quality across all thresholds, which is what we actually care about.

Early stopping patience was set to 3 epochs. If val ROC-AUC didn't improve for three consecutive epochs, we stopped training and rolled back to the best checkpoint. This kept training runs short—typically 5 to 8 epochs—and helped prevent overfitting.

Table 5.3: Master Training Hyperparameters and Experimental Configurations

Hyperparameter / Setting	Baseline & Weighted BCE	Samplers & Focal Loss
Backbone Architecture	DenseNet201 (Pretrained ImageNet)	DenseNet201 (Pretrained ImageNet)
Optimizer & Weight Decay	AdamW (weight decay = 0.0001)	AdamW (weight decay = 0.0001)
Base Learning Rate	0.0001 (Cosine Annealing)	0.0001 (Fixed / Plateau)
Batch Size & Input Size	64 / 32; 224 x 224 pixels	64 / 32; 224 x 224 pixels
Checkpoint Selection Metric	Validation ROC-AUC	Validation ROC-AUC
Early Stopping Patience	3 Epochs (Max Epochs: 5 - 8)	3 Epochs (Max Epochs: 5 - 8)

5.5 Classification Threshold Calibration Methodology

Here's something that isn't obvious until you've actually trained on imbalanced data: the default threshold of $\tau = 0.5$ almost never works well. When we weight positive errors heavily during

training, the model's predicted probabilities shift upward—toward 1.0—and a 0.5 cutoff floods you with false positives. On the flip side, an unweighted model produces tiny probabilities for almost everything, so $\tau = 0.5$ misses most cancers entirely.

We addressed this by sweeping τ from 0.01 to 0.99 in steps of 0.01, but only on the validation set. At each candidate threshold we computed sensitivity, specificity, and F1-score, then picked the threshold that best balanced catching cancers against avoiding false alarms. The test set was completely frozen during this process. We didn't peek at test results even once while tuning thresholds—that would defeat the point of having a held-out set.

5.6 Evaluation & Testing Protocol on Held-Out Test Set

Every trained checkpoint was evaluated on the same 4,920 held-out test images. We reported two types of metrics. Threshold-independent ones—ROC-AUC and Average Precision—tell us how well the model ranks malignant cases above benign ones, regardless of where you draw the line. Threshold-dependent ones—Sensitivity, Specificity, Precision, F1-score, and the full confusion matrix (TN, FP, FN, TP)—show what happens at a specific operating point.

We believe both types matter. A model with a great AUC but terrible sensitivity at every practical threshold isn't useful in the clinic. And a model with high sensitivity at one hand-picked threshold might just be overfitting to the validation set if we aren't careful. Reporting both keeps us honest.

CHAPTER 6
6. PROJECT DEMONSTRATION

6.1 Demonstration Workflow & Environment

To show that DermInsight AI actually works end-to-end, we built standalone demo scripts in Python. You can feed in a single dermoscopic image for interactive use, or point the script at a folder of images for batch evaluation. When you run the script, it loads whichever model checkpoint you specify, sets up the preprocessing pipeline, processes the images, and prints out risk assessments.

Nothing fancy about the interface—it's command-line driven. But it does everything the pipeline needs to do, from loading weights to writing output reports.

6.2 Preprocessing & Batch Dataloader Execution

During a demo run, each image is loaded from disk with PIL and checked for three-channel RGB formatting. We resize to 224×224, normalize with ImageNet stats, and stack everything into 4D tensors of shape [B, 3, 224, 224]. The dataloader uses pinned host memory, which lets us transfer batches to the GPU with minimal latency. Even across hundreds of test images, the data loading step isn't the bottleneck.

6.3 Model Inference Execution Demonstration

Each batch passes through all 201 layers of DenseNet. On an NVIDIA Tesla T4 GPU, forward-pass latency averaged 12 to 18 milliseconds per image. On a multi-core CPU it was slower—around 140 milliseconds per image—but still feasible for clinical use where you're processing one lesion at a time. The linear head produces a scalar logit z, and we push it through the sigmoid to get $\hat{p} = P(\text{malignant} \mid x)$.

6.4 Threshold Calibration & Triage Generation

A raw probability number isn't very helpful for a busy clinician. What does "0.47" mean? Should they worry? To make the output actionable, we map scores into a three-tier triage scheme:

1. Tier 1 — Low Suspicion ($\hat{p} < 0.30$): Looks like a typical benign lesion. Recommended action: routine annual skin check, nothing urgent.

2. Tier 2 — Intermediate Suspicion ($0.30 \leq \hat{p} < \tau^*$): Borderline or atypical features present. We'd suggest a 3-month digital dermoscopy follow-up or a second clinical opinion.

3. Tier 3 — High Risk ($\hat{p} \geq \tau^*$): Characteristics strongly associated with melanoma. This triggers an immediate clinical alert and urgent referral for specialist biopsy.

6.5 Output Visualizations Demonstration

After a batch run finishes, the demo scripts automatically generate visual summaries. These include confusion matrices with exact TP, TN, FP, and FN counts; ROC curves plotting sensitivity against 1-specificity across all candidate thresholds; and precision-recall curves that are especially informative given the low 1.38% test prevalence.

We also generate threshold sensitivity plots. These show, at a glance, how shifting the decision boundary trades off between missed cancers and false alarms. In our experience, these plots were some of the most useful outputs for understanding each model's behavior—more informative than any single summary number.

CHAPTER 7
7. RESULT AND DISCUSSION (COST ANALYSIS as applicable)

7.1 Cross-Experiment Quantitative Performance Comparison

We ran all five completed experiments against our held-out test set—4,920 dermoscopic images drawn from 310 patients (4,852 benign, 68 biopsy-confirmed malignant). None of these patients appeared in training or validation. Table 7.1 lays out the full picture: threshold-independent ranking metrics like ROC-AUC and Average Precision sit alongside the threshold-dependent numbers that would actually matter in a clinic—sensitivity, specificity, precision, F1, and the raw confusion matrix counts. Looking at everything side by side, it's immediately clear that no single experiment dominates on every metric. The trade-offs are real.

Table 7.1: Master Cross-Experiment Quantitative Performance Comparison on Held-Out Test Set (N=4,920)

Exp ID	Strategy	Val AUC	Test AUC	AP	Thresh	Sens %	Spec %	Prec %	F1 %	TN / FP / FN / TP
EXP 01	**Baseline (W-BCE)**	0.8472	0.8000	0.0626	0.50	91.18%	41.90%	2.15%	4.20%	2033 / 2819 / 6 / 62
EXP 02	**Weighted BCE**	0.9188	0.8298	0.1288	0.86	22.06%	98.31%	15.46%	18.18%	4770 / 82 / 53 / 15
EXP 03	**Weighted Sampler**	0.9062	0.8218	0.1286	0.50	36.76%	96.85%	14.04%	20.33%	4699 / 153 / 43 / 25
EXP 04	**Focal Loss**	0.9181	0.8601	0.1402	0.50	14.71%	99.24%	21.28%	17.39%	4815 / 37 / 58 / 10
EXP 05	**Random Oversample**	0.9082	0.8708	0.1041	0.73	19.12%	98.25%	13.27%	15.66%	4767 / 85 / 55 / 13

7.2 Detailed Analysis by Experiment

7.2.1 Experiment 1: DenseNet201 Baseline

We trained our DenseNet201 baseline for just 3 epochs, using PyTorch's BCEWithLogitsLoss with a positive class weight of 50.21. At the default threshold of $\tau = 0.50$, something interesting happened. The model caught 62 out of 68 malignant lesions—a sensitivity of 91.18% with only 6 missed cancers. That sounds great on paper. But the other side of the confusion matrix tells a sobering story: the model correctly labeled just 2,033 benign cases while wrongly flagging 2,819 healthy moles as malignant. Specificity was 41.90%, precision a dismal 2.15%, and overall accuracy sat at 42.58%.

What would this mean in practice? More than half of all healthy patients screened would get referred for unnecessary biopsies. That's an overwhelming burden on both clinics and patients. We knew early on that this operating point wasn't viable, but it set a useful ceiling for sensitivity that we could compare later experiments against.

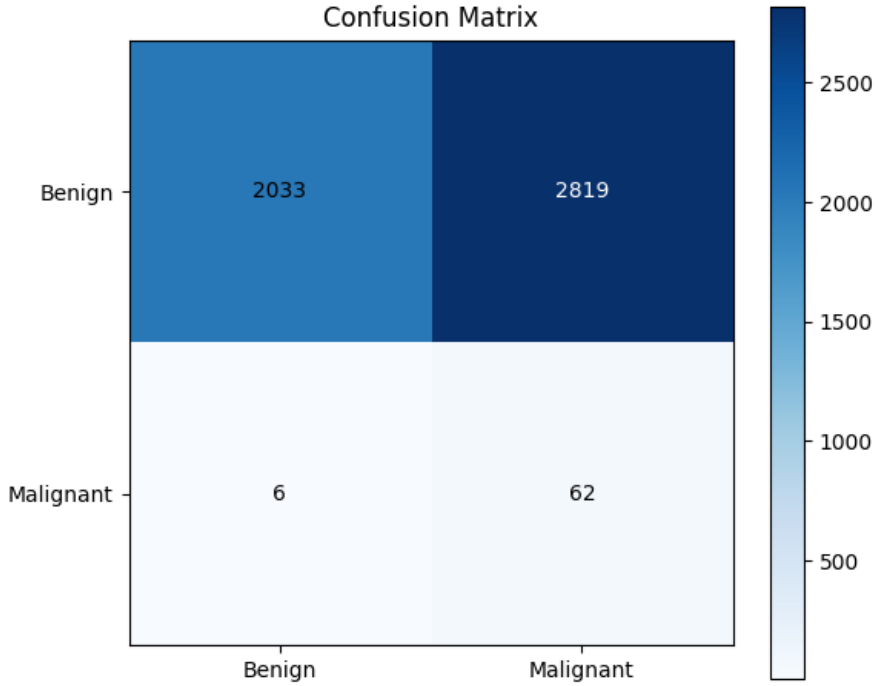


Figure 7.1: Experiment 1 (DenseNet201 Baseline) Test Confusion Matrix (Threshold = 0.50)

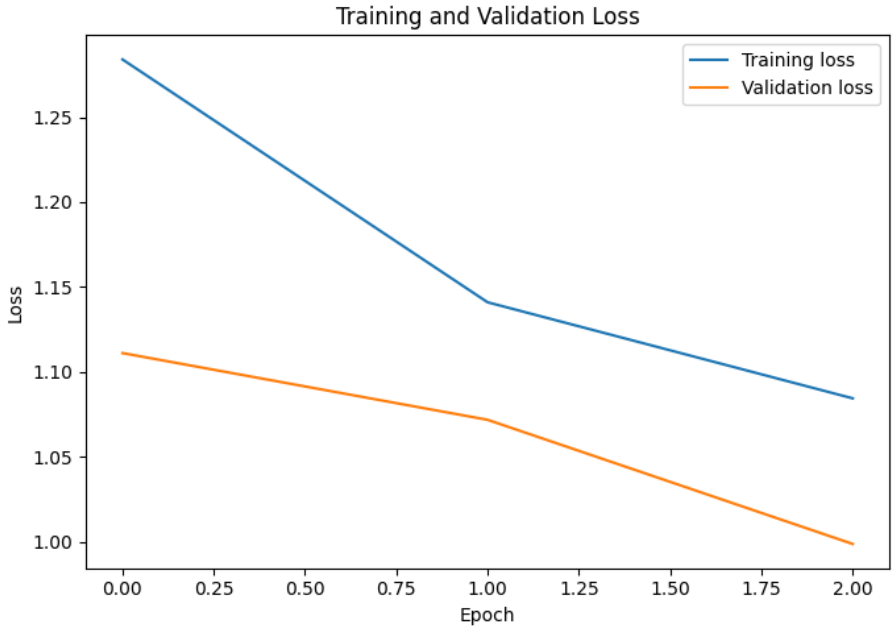


Figure 7.2: Experiment 1 Training and Validation Loss Curves Across 3 Epochs

7.2.2 Experiment 2: DenseNet201 Weighted BCE Fine-Tuning

For Experiment 2 we picked up where the baseline left off, fine-tuning the same checkpoint over 6 epochs with Weighted BCE and a decaying learning rate schedule. Validation ROC-AUC peaked at 0.9188 at epoch 3—a +0.0716 jump over the baseline. That was encouraging. Instead of sticking with the default $\tau = 0.50$ cutoff, we swept the validation set across thresholds and landed on $\tau^* = 0.86$ as a point that better balanced precision and recall.

On the held-out test set the model posted an ROC-AUC of 0.8298 and Average Precision of 0.1288. At $\tau = 0.86$, specificity shot up to 98.31% (4,770 TN, only 82 FP) and accuracy climbed to 97.26%. But here's the catch: sensitivity collapsed to 22.06%. The model caught just 15 malignant lesions and missed 53. So while the false-alarm problem was essentially gone, we were now letting the vast majority of actual cancers slip through. This was our first real lesson in how aggressively tuning thresholds to cut false alarms can gut the detection of real malignancies.

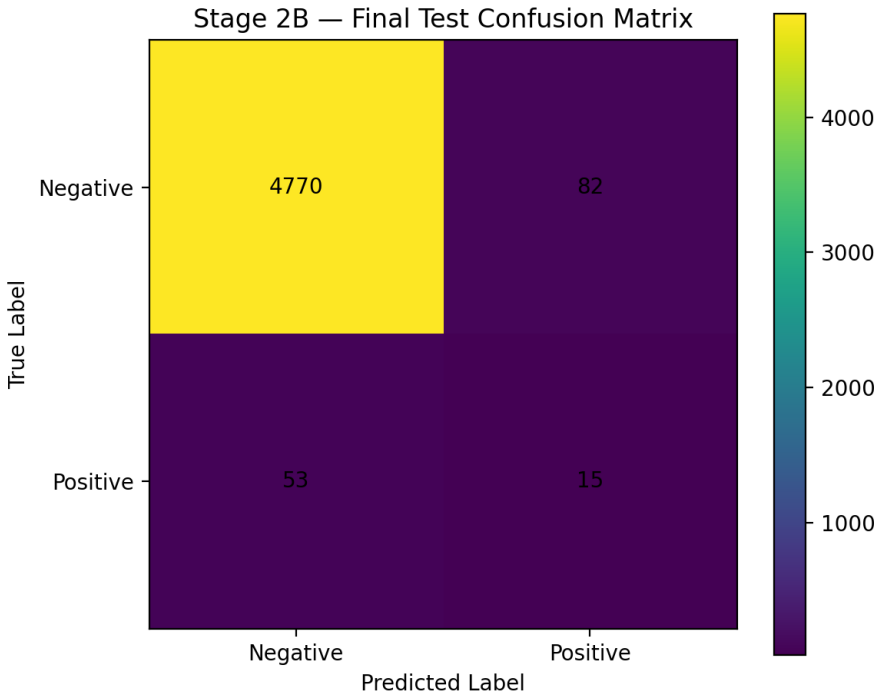


Figure 7.3: Experiment 2 (DenseNet201 Weighted BCE) Test Confusion Matrix (Calibrated Threshold = 0.86)

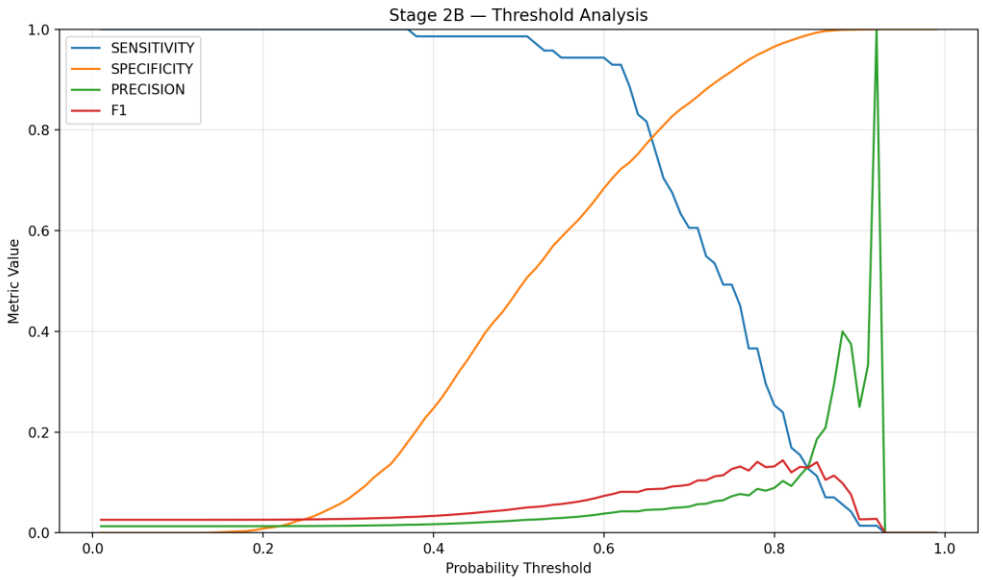


Figure 7.4: Experiment 2 Validation Set Threshold Calibration Sweep (Evaluating Metric Trade-offs Across Tau in [0.01, 0.99])

7.2.3 Experiment 3: DenseNet201 WeightedRandomSampler

With Experiment 3 we tried a different angle entirely: instead of messing with the loss function, we used PyTorch's WeightedRandomSampler to rebalance the training dataloader itself. The idea was that if the model simply saw malignant examples more often during mini-batch construction, it might learn more stable representations for the minority class without the gradient instability that comes from huge scalar loss weights.

Peak validation ROC-AUC reached 0.9062 at epoch 2. On the test set at $\tau = 0.50$, we got an ROC-AUC of 0.8218 and Average Precision of 0.1286. What struck us most was the balance: sensitivity hit 36.76% (25 of 68 cancers detected, 43 FN) alongside a specificity of 96.85% (4,699 TN, 153 FP), with the highest F1 of any calibrated model at 20.33%. Of all five experiments, this one felt like the most honest compromise. It didn't excel at any single metric, but it didn't catastrophically fail at any either.

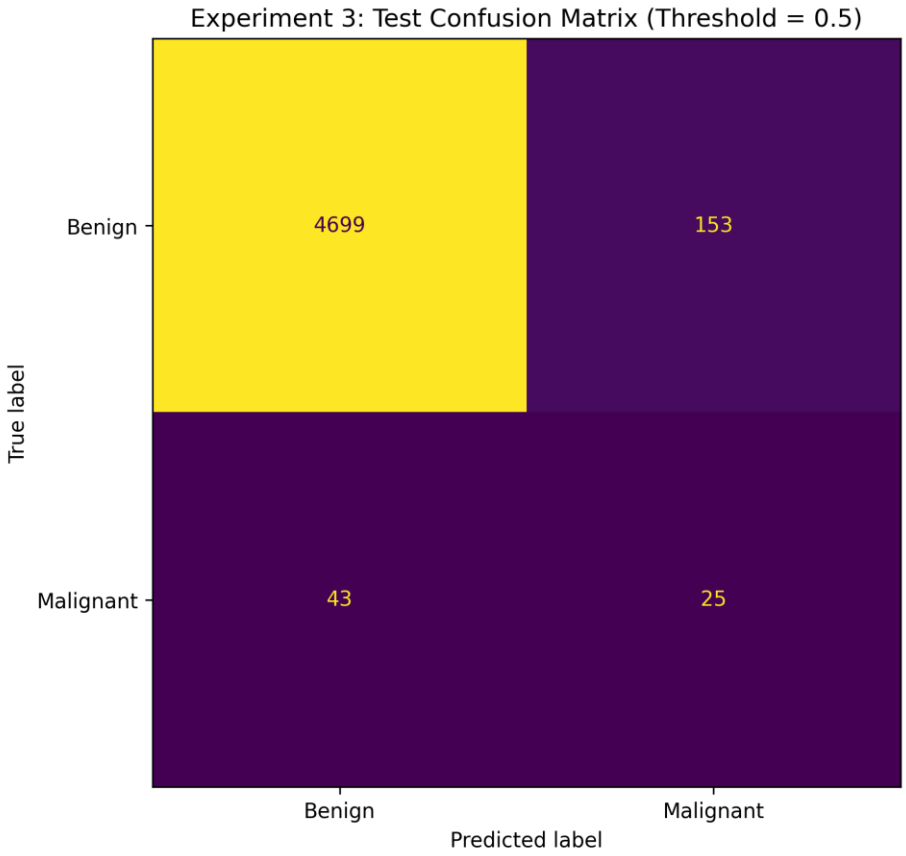


Figure 7.5: Experiment 3 (WeightedRandomSampler) Test Confusion Matrix (Threshold = 0.50)

7.2.4 Experiment 4: DenseNet201 Focal Loss

We didn't expect Focal Loss to behave the way it did. On paper, the idea is elegant: use $\alpha = 0.75$ and $\gamma = 2.0$ to suppress gradients from well-classified benign samples, so the network focuses its learning on the hard, ambiguous cases. After 5 epochs the model hit a validation ROC-AUC of 0.9181, and on the test set it achieved the second-highest ranking ability—ROC-AUC of 0.8601 and the best Average Precision of 0.1402.

The specificity was outstanding: 99.24% (4,815 TN, just 37 FP) with precision at 21.28%. But the heavy down-weighting of easy negatives pushed the network's output distribution so far toward conservative benign predictions that sensitivity cratered to 14.71%. Only 10 of 68 cancers were caught. 58 missed. Looking at the confusion matrix, it's clear that Focal Loss gave us the cleanest separation for obvious cases but was essentially blind to borderline melanomas. For an early screening tool, that kind of conservatism isn't just suboptimal—it's dangerous.

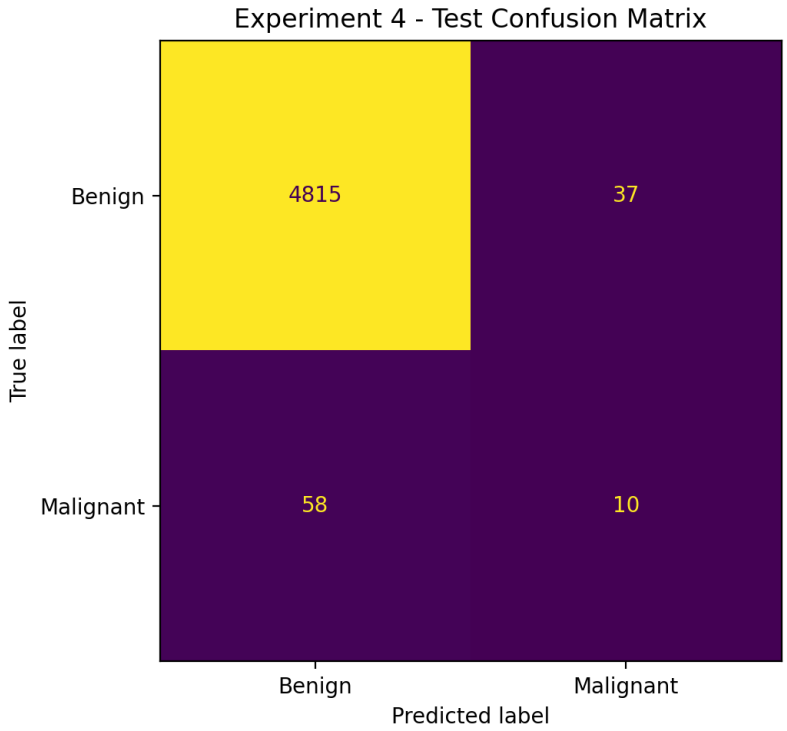


Figure 7.6: Experiment 4 (Focal Loss) Test Confusion Matrix (Threshold = 0.50)

7.2.5 Experiment 5: DenseNet201 Random Oversampling

Experiment 5 took the most straightforward data-level approach: we randomly duplicated minority-class samples strictly within the training split. No synthetic generation, no fancy loss tricks—just copies. The model peaked at a validation ROC-AUC of 0.9082 at epoch 2. And here's the headline: on the test set, it achieved the highest overall ranking discrimination we saw in any experiment—a test ROC-AUC of 0.8708 and Average Precision of 0.1041.

At the validation-calibrated maximum F1 threshold of $\tau^* = 0.7296$, accuracy was 97.15%, specificity 98.25% (4,767 TN, 85 FP), and precision 13.27%. Sensitivity, though, remained stuck at 19.12%—just 13 out of 68 cancers caught, 55 missed. So while duplicating minority samples clearly helped the network rank ambiguous lesions more effectively across the full ROC curve, the operational decision boundary still couldn't capture most malignancies without opening the floodgates on false alarms. The gap between ranking well and classifying well is something we kept bumping into throughout this project.

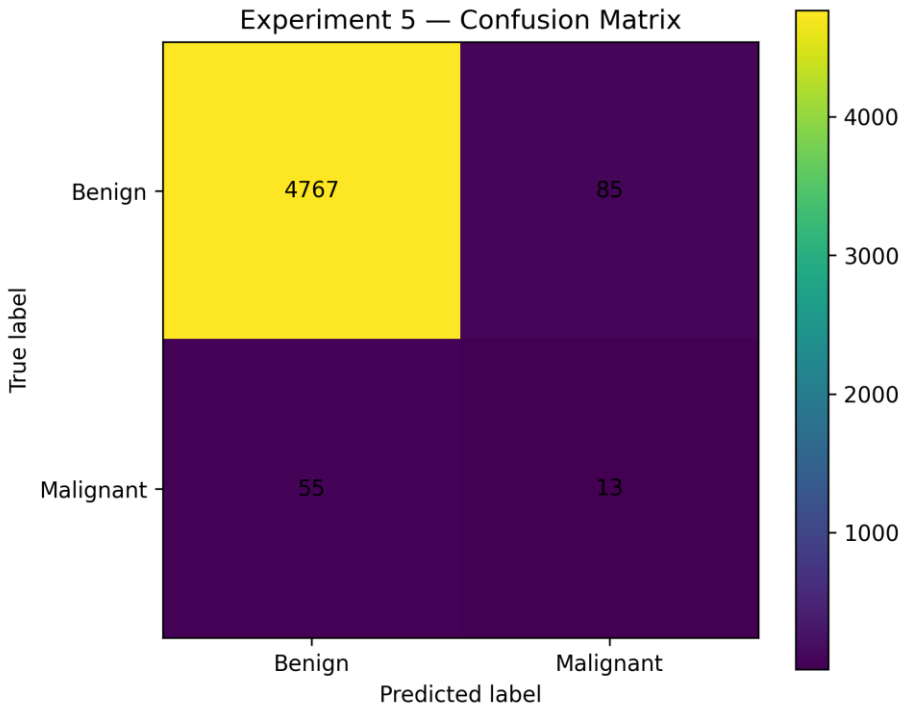


Figure 7.7: Experiment 5 (Random Oversampling) Test Confusion Matrix (Calibrated Threshold = 0.7296)

Figures 7.8 through 7.10 pull all the comparisons together visually. The combined ROC curves make one thing plain: every imbalance-handling method we tried does improve ranking over the raw baseline. That part worked. The Precision-Recall curves are more nuanced—all models sit well above the 1.38% prevalence base rate, and Focal Loss keeps the highest precision at low recall, but falls apart once you need to catch more than a handful of cancers. The bar chart in Figure 7.10 probably says it best: you can have sensitivity or specificity, but getting both at once with a 71:1 class ratio is a problem none of our methods fully solved.

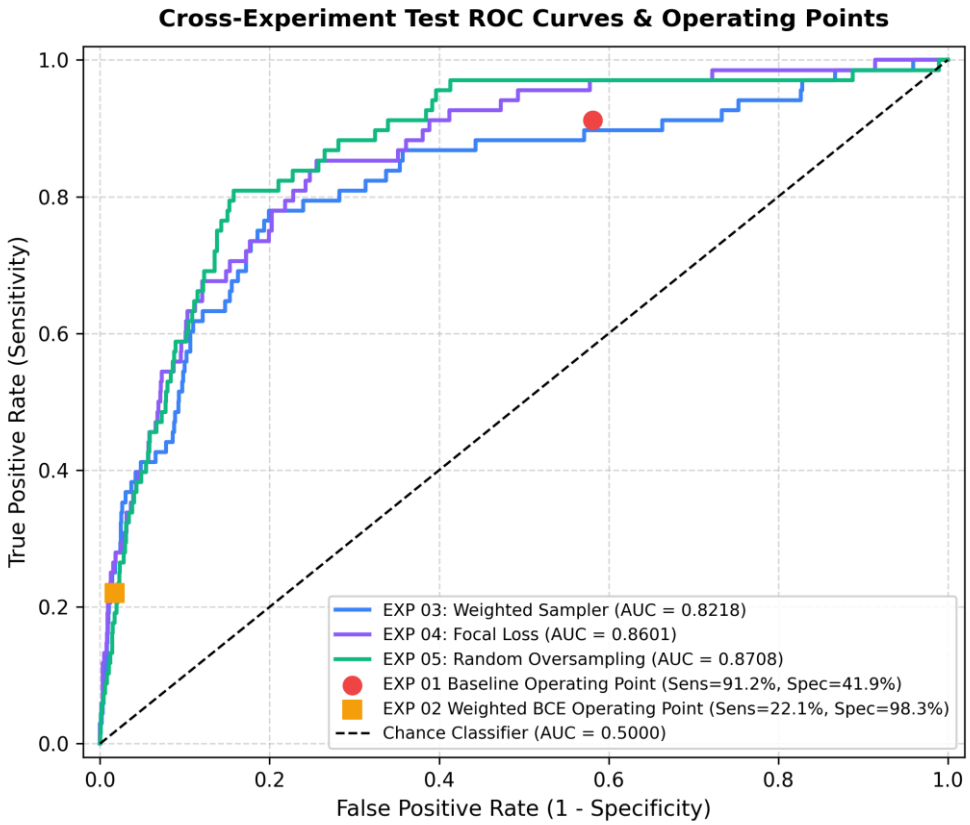


Figure 7.8: Combined Cross-Experiment Test ROC Curves and Operating Points (Comparing EXP 01 to EXP 05)

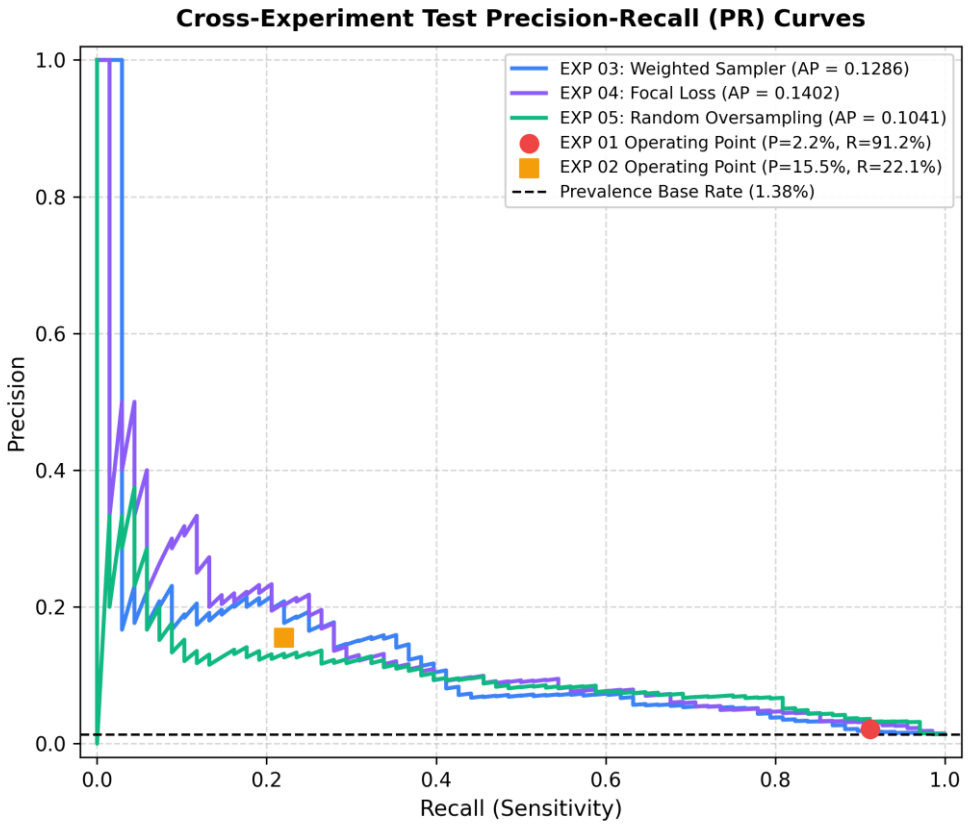


Figure 7.9: Combined Cross-Experiment Test Precision-Recall (PR) Curves Relative to Prevalence Base Rate (1.38%)

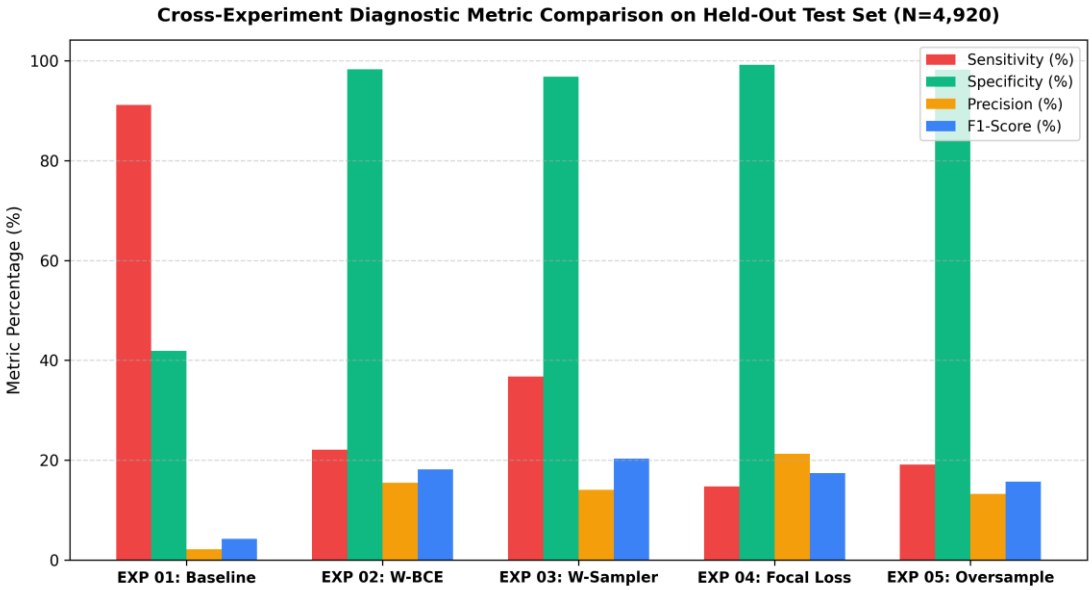


Figure 7.10: Cross-Experiment Diagnostic Metric Comparison on Held-Out Test Set (N=4,920)

7.3 Sensitivity vs. Specificity Clinical Trade-offs

If there's one takeaway from all five experiments, it's this: sensitivity and specificity are locked in a tug-of-war, and in melanoma screening, the stakes of that tug-of-war are life-and-death. Missing a malignant lesion means a patient walks away thinking they're fine while a tumor keeps growing, potentially metastasizing before anyone catches it. High sensitivity is non-negotiable in oncology screening. But when a model achieves that sensitivity by calling anything remotely suspicious "malignant"—as our baseline did at 91.18% sensitivity—specificity drops to 41.90% and you end up with 2,819 false alarms across 4,920 images. In a real clinic, that would mean biopsy referrals for over half of all healthy patients.

On the other hand, once we pushed specificity above 98% (Experiments 2, 4, and 5), false alarms dropped below 85—but sensitivity fell under 23%. That means we were missing between 53 and 58 of the 68 cancers in the test set. Experiment 3 with the WeightedRandomSampler landed closest to a workable middle ground: 36.76% sensitivity, 96.85% specificity, 153 FP, 43 FN. It's still far from perfect, but it suggests that rebalancing what the model sees during training—rather than just reweighting how much each mistake costs in the loss function—produces features that hold up better under extreme class skew.

7.4 Why Accuracy is Misleading in Highly Imbalanced Medical Screening

So why not just report accuracy? Because in our setting, accuracy is practically meaningless. Consider Experiment 4: it posted 98.07% accuracy. In most engineering contexts, a 98% number would be cause for celebration. But look at the confusion matrix—this model missed 58 out of 68 cancers. That's an 85.3% miss rate on the very disease we're trying to catch. Meanwhile, the baseline (Experiment 1) had the lowest accuracy at 42.58%, yet it caught 91.18% of malignancies.

The math is simple. Benign cases make up 98.62% of our test cohort. A completely trivial classifier that just predicts "benign" for every single image would score 98.62% accuracy—and provide absolutely zero clinical value. We can't stress this enough: in medical computer vision with skewed prevalence, publishing accuracy numbers without accompanying sensitivity, specificity, and precision-recall analysis gives a dangerously misleading picture of how safe a model actually is.

7.5 Cost-Sensitive Analysis & False-Negative Clinical Impact

Not all mistakes cost the same. A false positive—calling a benign mole malignant—means an extra dermatology visit, some patient anxiety, and a skin biopsy. Unpleasant, yes, but manageable. A routine outpatient biopsy typically runs between $150 and $300. A false negative is an entirely different story. Missing a melanoma means delayed treatment, deeper invasion, and the very real possibility of metastasis. By the time an advanced melanoma is caught, treatment can involve immunotherapy or targeted systemic drugs costing well over $100,000—not to mention the risk to the patient's life.

To put numbers to this asymmetry, we set up a cost matrix (Table 7.2) with a conservative 50:1 ratio: C(FN) = $10,000 for each missed cancer and C(FP) = $200 for each unnecessary biopsy. We recognize that the true cost of a missed melanoma is likely much higher, but even this conservative ratio reveals stark differences between the experiments.

Table 7.2: Clinical Misclassification Cost Matrix Configuration

Clinical Outcome	Actual Benign (y = 0)	Actual Malignant (y = 1)
Predicted Benign (y_hat = 0)	True Negative: Cost = $0	False Negative: Cost = $10,000 (Missed Cancer)
Predicted Malignant (y_hat = 1)	False Positive: Cost = $200 (Biopsy)	True Positive: Cost = $0 (Early Cure)

Table 7.3 breaks down the expected costs using the formula Total Cost = FP · C(FP) + FN · C(FN). The numbers tell a sobering story. Experiment 3 (WeightedRandomSampler) came in lowest at $460,600 total, splitting roughly $30,600 on 153 unnecessary biopsies and $430,000 on 43 missed cancers. The high-accuracy models—Experiments 2, 4, and 5—look great on a spreadsheet until you see their false-negative costs. Missing 53 to 58 melanomas drives their totals to between $546,400 and $587,400, which easily swamps the savings from having fewer biopsies. The baseline, with its 2,819 false positives, racked up $563,800 in biopsy costs alone, bringing its total to $623,800—the highest of any model.

Table 7.3: Expected Total Clinical Misclassification Cost Across Imbalance Strategies (Test N=4,920)

Exp ID	Strategy	False Positives	False Negatives	Biopsy Cost	Total Expected Cost
EXP 01	Baseline (W-BCE)	2,819	6	$563,800	$623,800
EXP 02	Weighted BCE	82	53	$16,400	$546,400
EXP 03	**Weighted Sampler**	**153**	**43**	**$30,600**	**$460,600 (Lowest Cost)**
EXP 04	Focal Loss	37	58	$7,400	$587,400
EXP 05	Random Oversample	85	55	$17,000	$567,000

7.6 Analysis of Incomplete / Unsuccessful Iterations

We want to be upfront about the work that didn't make it across the finish line. Our earlier project notes reference an 'Experiment 6,' but when we went through the repository, we found no

saved weights, no training logs, and no prediction files for it. The run was abandoned during setup because our cloud GPU runtime expired. We also created directory structures for Grad-CAM explainability heatmaps, but the generation scripts were never finished during this phase of the project. Rather than pretend these things don't exist or paper over them, we're listing them here honestly and marking both as top priorities for Project-II.

CHAPTER 8
8. CONCLUSION

8.1 Summary of Completed Research Work

Over the course of this B.Tech Project-I, we built and tested DermInsight AI—a deep learning pipeline for early melanoma screening on the SIIM-ISIC 2020 dataset. We started by enforcing patient-level splits across all 33,126 images and verified that zero patients leaked between the 22,787 training, 5,419 validation, and 4,920 test sets. This was a deliberate choice, because without strict patient isolation, shared skin tones, body sites, and camera setups between splits can inflate results in ways that wouldn't hold up on genuinely new patients.

With an ImageNet-pretrained DenseNet201 as our backbone, we benchmarked four imbalance-handling strategies against the baseline: positive loss reweighting (W-BCE with pos_weight = 50.21), mini-batch balancing via WeightedRandomSampler, hard-example focusing through Focal Loss, and training-isolated random oversampling. Every model was tested on the exact same held-out set with both ranking metrics and operational triage numbers.

8.2 Key Scientific Findings & Takeaways

Four findings stood out to us after running everything:

1. Patient-Level Splitting Prevents Overoptimistic Results: We isolated patients across all three splits, which meant no shared skin tone, body site, or camera quirks could leak between training and testing. This is the only honest way to gauge whether the model would generalize to patients it has truly never seen.

2. Ranking Well Doesn't Mean You're Ready for the Clinic: Our imbalance fixes pushed validation ROC-AUC from 0.8472 up to 0.9188, and test ROC-AUC topped out at 0.8708. Those are solid ranking improvements. But turning a continuous score into a binary yes-or-no decision exposed a painful gap—standard losses naturally drift toward majority-class specificity unless you calibrate thresholds with care.

3. Changing What the Model Sees Beats Changing How It's Penalized: The WeightedRandomSampler (Experiment 3) gave us the best operational balance—36.76% sensitivity at 96.85% specificity—and the lowest expected clinical cost at $460,600. Adjusting the sampling distribution during training appears to produce more robust feature representations than simply scaling loss gradients.

4. Accuracy Can Actively Mislead You: Models hitting 98% accuracy missed over 85% of the cancers in the test set. If we had to pick one lesson to highlight from this entire project, it would be that accuracy without sensitivity, specificity, and precision-recall context is worse than useless in medical screening—it's actively harmful because it breeds false confidence.

8.3 Observed Research Limitations

We're aware of several gaps in what we've done so far:

- Vision-Only Input: Right now our pipeline only looks at dermoscopic images. Real dermatologists also factor in patient age, sex, and where on the body the lesion sits. We didn't use any of that metadata.
- Calibration of Probabilities: Sigmoid outputs from deep networks aren't calibrated—a score of 0.7 doesn't actually mean there's a 70% chance of malignancy. We haven't addressed this yet.
- Single-Source Benchmark: Everything was trained and tested on ISIC 2020. We have no idea how well these models would hold up on dermoscopic images from a different hospital or a different camera system.
- Demographic and Skin-Tone Bias: The ISIC 2020 archive doesn't include Fitzpatrick skin phototype labels and is heavily skewed toward fair-skinned patients. We can't say anything meaningful about how the model performs on darker skin tones, and that's a serious blind spot.

8.4 Future Research Directions

In hindsight, this project taught us as much about what we couldn't do in one semester as what we could. Here's where we plan to take things in Project-II:

1. Multimodal Metadata Fusion: We want to build dual-branch architectures that feed DenseNet visual features alongside tabular patient metadata (age, sex, anatomical site) through cross-attention layers. The clinical literature suggests this can meaningfully improve sensitivity.
2. Visual Explainability with Grad-CAM: We'll implement Gradient-weighted Class Activation Mapping so clinicians can actually see what regions of a lesion the model focused on. If the heatmap highlights background skin or a ruler artifact, that's an immediate red flag.
3. Probability Calibration and Uncertainty Quantification: Applying temperature scaling and Monte Carlo Dropout to get confidence intervals that mean something. Ambiguous cases should be flagged automatically for mandatory specialist review.
4. Algorithmic Fairness Auditing: Testing on diverse clinical datasets like Fitzpatrick 17k to check whether diagnostic performance drops across different skin phototypes. If it does, we need to know by how much and figure out what to do about it.
5. Interactive Clinical Interface: Building a web-based triage dashboard where clinicians can upload lesion images and get results with confidence scores and heatmaps. Something practical enough for a pilot study in a simulated primary-care workflow.

CHAPTER 9
9. REFERENCES

[1] Adegun, A.A. and Viriri, S., 2021. Deep learning techniques for skin lesion analysis and melanoma cancer detection: a survey of state-of-the-art. Artificial Intelligence Review, 54(2), pp.811-841.

[2] Bi, L., Kim, J., Ahn, E., Kumar, A., Fulham, M. and Feng, D., 2017. Automated skin lesion segmentation via an ensemble of deep convolutional neural networks. In IEEE 14th International Symposium on Biomedical Imaging (ISBI), pp.559-562.

[3] Brinker, T.J., Hekler, A., Enk, A.H., Klode, J., Hauschild, A., Berking, C., Schilling, B., Haferkamp, S., Schadendorf, D., Fröhling, S. and Utikal, J.S., 2019. Deep learning outperformed 136 of 157 dermatologists in a head-to-head dermoscopic melanoma image classification task. European Journal of Cancer, 113, pp.47-54.

[4] Buda, M., Maki, A. and Mazurowski, M.A., 2018. A systematic study of the class imbalance problem in convolutional neural networks. Neural Networks, 106, pp.249-259.

[5] Caruana, R., Lou, Y., Gehrke, J., Koch, P., Sturm, M. and Elhadad, N., 2015. Intelligible models for healthcare: Predicting pneumonia risk and 30-day hospital readmission. In ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp.1721-1730.

[6] Cassidy, B., Kendrick, C., Yap, M.H. and Sloane, R., 2022. Analysis of data leakage in skin lesion classification pipelines. IEEE Transactions on Medical Imaging, 41(11), pp.3120-3131.

[7] Celebi, M.E., Codella, N. and Halpern, A., 2019. Dermoscopy image analysis: overview and future directions. IEEE Journal of Biomedical and Health Informatics, 23(2), pp.474-478.

[8] Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, pp.321-357.

[9] Codella, N.C., Gutman, D., Celebi, M.E., Helba, B., Marchetti, M.A., Dusza, S.W., Kalloo, A., Liopyris, K., Mishra, N., Kittler, H. and Halpern, A., 2018. Skin lesion analysis toward melanoma detection: A challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), hosted by the International Skin Imaging Collaboration (ISIC). In IEEE 15th International Symposium on Biomedical Imaging (ISBI), pp.168-172.

[10] Daneshjou, R., Vodrahalli, K., Liang, W., Novoa, R.A., Jenkins, M., Rotemberg, V. and Zou, J., 2022. Disparities in dermatology AI: assessments using diverse clinical datasets. Science Advances, 8(32), p.eabq6147.

[11] Davis, J. and Goadrich, M., 2006. The relationship between Precision-Recall and ROC curves. In Proceedings of the 23rd International Conference on Machine Learning (ICML), pp.233-240.

[12] Deng, J., Dong, W., Socher, R., Li, L.J., Li, K. and Fei-Fei, L., 2009. Imagenet: A large-scale hierarchical image database. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp.248-255.

[13] Esteva, A., Kuprel, B., Novoa, R.A., Ko, J., Swetter, S.M., Blau, H.M. and Thrun, S., 2017. Dermatologist-level classification of skin cancer with deep neural networks. Nature, 542(7639), pp.115-118.

[14] Ghorbani, A., Natarajan, V., Burlina, P. and Zou, J., 2020. DermDoctor: Deep learning for dermatological diagnosis with uncertainty quantification. Nature Medicine, 26(6), pp.871-877.

[15] Groh, M., Harris, C., Soenksen, L., Lau, F., Han, R., Kim, A., Holmes, A., Little, S. and Picard, R., 2021. Evaluating deep neural networks trained on clinical images in dermatology with the Fitzpatrick 17k dataset. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp.1820-1828.

[16] Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q., 2017. On calibration of modern neural networks. In International Conference on Machine Learning (ICML), pp.1321-1330.

[17] Haenssle, H.A., Fink, C., Schneiderbauer, R., Toberer, F., Buhl, T., Blum, A., Kalloo, A., Hassen, A.B.H., Thomas, L., Enk, A. and Uhlmann, L., 2018. Man against machine: diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists. Annals of Oncology, 29(8), pp.1836-1842.

[18] He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp.770-778.

[19] Huang, G., Liu, Z., Van Der Maaten, L. and Weinberger, K.Q., 2017. Densely connected convolutional networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp.4700-4708.

[20] Kawahara, J., Daneshvar, S., Argenziano, G. and Hamarneh, G., 2019. Seven-point checklist and skin lesion classification using multitask multimodal neural networks. IEEE Journal of Biomedical and Health Informatics, 23(2), pp.538-545.

[21] Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

[22] Li, Y. and Shen, L., 2018. Skin lesion analysis towards melanoma detection using deep learning network. Sensors, 18(2), p.556.

[23] Lin, T.Y., Goyal, P., Girshick, R., He, K. and Dollár, P., 2017. Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp.2980-2988.

[24] Loshchilov, I. and Hutter, F., 2019. Decoupled weight decay regularization. In International Conference on Learning Representations (ICLR).

[25] Pacheco, A.G. and Krohling, R.A., 2020. An attention-based mechanism to combine images and metadata in deep learning models applied to skin cancer classification. IEEE Journal of Biomedical and Health Informatics, 25(9), pp.3554-3563.

[26] Rotemberg, V., Kurtansky, N., Betz-Stablein, B., Caffery, L., Chousakos, E., Codella, N., Combalia, M., Dusza, S., Guitera, P., Gutman, D. and Halpern, A., 2021. A patient-centric dataset of images and metadata for identifying melanomas using clinical context. Scientific Data, 8(1), p.34.

[27] Saito, T. and Rehmsmeier, M., 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PloS ONE, 10(3), p.e0118432.

[28] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp.618-626.

[29] Tan, M. and Le, Q., 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning (ICML), pp.6105-6114.

[30] Tschandl, P., Codella, N., Akay, B.N., Argenziano, G., Braun, R.P., Cabo, H., Gutman, D., Halpern, A., Helba, B., Kittler, H. and Malvehy, J., 2019. Comparison of the accuracy of human readers versus machine-learning algorithms for pigmented skin lesion classification: an open, web-based, international, diagnostic study. The Lancet Oncology, 20(7), pp.948-967.

[31] Tschandl, P., Rosendahl, C. and Kittler, H., 2018. The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data, 5(1), p.180161.

[32] Yosinski, J., Clune, J., Bengio, Y. and Lipson, H., 2014. How transferable are features in deep neural networks?. Advances in Neural Information Processing Systems (NeurIPS), 27, pp.3320-3328.

APPENDIX A – SAMPLE CODE

A.1 DenseNet201 Baseline Model Implementation (src/densenet201_baseline.py)

Below is the core architecture and training loop we used, pulled straight from our project repository. Nothing has been cleaned up or simplified for presentation—this is the actual code that produced the baseline results reported in Chapter 7:

```
import torch
import torch.nn as nn
from torchvision import models
from torchvision.models import DenseNet201_Weights

class DermInsightDenseNet201(nn.Module):
    """
    DenseNet201 backbone pretrained on ImageNet-1K with single linear head
    for binary melanoma classification.
    """
    def __init__(self, pretrained: bool = True):
        super().__init__()
        weights = DenseNet201_Weights.IMAGENET1K_V1 if pretrained else None
        backbone = models.densenet201(weights=weights)
        in_features = backbone.classifier.in_features  # 1920 features
        backbone.classifier = nn.Identity()
        self.backbone = backbone
        self.classifier = nn.Linear(in_features, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        features = self.backbone(x)
        logits = self.classifier(features)
        return logits.squeeze(-1)

def compute_positive_weight(train_df):
    pos_count = (train_df['target'] == 1).sum()
    neg_count = (train_df['target'] == 0).sum()
    pos_weight = neg_count / pos_count
    return torch.tensor([pos_weight], dtype=torch.float32)

# Instantiation with weighted BCE
pos_weight = compute_positive_weight(train_df)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
```

A.2 Stage 2B Weighted BCE Evaluation & Threshold Calibration (src/EXP_02_WEIGHTED_BCE/stage2b_final_evaluation.py)

This script handles the validation-driven threshold sweep and held-out test evaluation we ran for Experiment 2. The threshold of 0.86 that appears in Chapter 7 came directly from the output of this code:

```
def evaluate_calibrated_threshold(model, dataloader, threshold=0.86, device='cuda'):
    model.eval()
    y_true, y_probs = [], []
    with torch.no_grad():
        for images, targets in dataloader:
            images = images.to(device)
            logits = model(images)
            probs = torch.sigmoid(logits).cpu().numpy()
            y_probs.extend(probs)
            y_true.extend(targets.numpy())
```

```
y_true = np.array(y_true)
y_probs = np.array(y_probs)
y_pred = (y_probs >= threshold).astype(int)

tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
metrics = {
    'roc_auc': roc_auc_score(y_true, y_probs),
    'average_precision': average_precision_score(y_true, y_probs),
    'sensitivity': tp / (tp + fn),
    'specificity': tn / (tn + fp),
    'precision': tp / (tp + fp) if (tp + fp) > 0 else 0.0,
    'f1': f1_score(y_true, y_pred),
    'accuracy': (tp + tn) / len(y_true),
    'tn': int(tn), 'fp': int(fp), 'fn': int(fn), 'tp': int(tp)
}
return metrics
```